



(21) 申请号 202610856083.7

(22) 申请日 2026.06.15

(71) 申请人 上海燧原科技股份有限公司

地址 201306 上海市浦东新区中国(上海)
自由贸易试验区临港新片区业盛路
188号A-522室

(72) 发明人 周荣华 卞劲松 林川 张文平

(74) 专利代理机构 北京品源专利代理有限公司
11332

专利代理师 岳晓萍

(51) Int. Cl.

G06F 11/3668 (2025.01)

G06F 11/22 (2006.01)

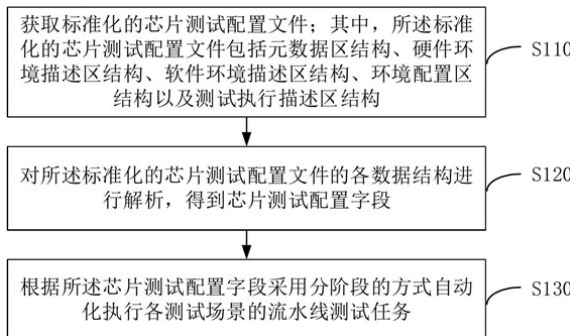
权利要求书3页 说明书34页 附图5页

(54) 发明名称

一种芯片测试方法、装置、电子设备、存储介质及程序

(57) 摘要

本发明实施例公开了一种芯片测试方法、装置、电子设备、存储介质及程序,其中,方法包括:获取标准化的芯片测试配置文件;其中,所述标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构;对所述标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段;根据所述芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流水线测试任务。本发明实施例的技术方案能够快速且精确地复现芯片的专用测试场景。



1. 一种芯片测试方法,其特征在于,包括:

获取标准化的芯片测试配置文件;其中,所述标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构;

对所述标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段;

根据所述芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流水线测试任务;

所述对所述标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段,包括:

对所述标准化的芯片测试配置文件的各数据结构进行解析和验证,得到初始化芯片测试配置字段;

从所述初始化芯片测试配置字段提取芯片领域专用字段;

对所述芯片领域专用字段进行结构化配置,得到所述芯片测试配置字段。

2. 根据权利要求1所述的方法,其特征在于,所述获取标准化的芯片测试配置文件,包括:

采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息;

根据所述实时状态关联信息生成标准化的芯片测试配置文件。

3. 根据权利要求2所述的方法,其特征在于,所述根据所述实时状态关联信息生成标准化的芯片测试配置文件,包括:

根据所述实时状态关联信息填充所述芯片测试配置文件的预设数据结构,得到初始芯片测试配置文件;

对所述初始芯片测试配置文件中的配置数据进行规范化验证;

在确定所述初始芯片测试配置文件通过验证的情况下,将所述初始芯片测试配置文件进行序列化处理,得到所述标准化的芯片测试配置文件。

4. 根据权利要求3所述的方法,其特征在于,所述根据所述实时状态关联信息填充所述芯片测试配置文件的预设数据结构,包括:

从所述实时状态关联信息中提取元数据字段,并根据所述元数据字段填充所述芯片测试配置文件的元数据区结构;

从所述实时状态关联信息中提取芯片硬件环境检测字段,并根据所述芯片硬件环境检测字段填充所述芯片测试配置文件的硬件环境描述区结构;

从所述实时状态关联信息中提取芯片软件环境字段,并根据所述芯片软件环境字段填充所述芯片测试配置文件的软件环境描述区结构;

从所述实时状态关联信息中提取环境配置字段,并根据所述环境配置字段填充所述芯片测试配置文件的环境配置区结构;

从所述实时状态关联信息中提取测试执行字段,并根据所述测试执行字段填充所述芯片测试配置文件的测试执行描述区结构。

5. 根据权利要求1所述的方法,其特征在于,所述对所述标准化的芯片测试配置文件的各数据结构进行解析和验证,包括:

根据所述标准化的芯片测试配置文件的目标数据结构解析得到芯片检测命令字段以

及驱动安装命令字段；

根据当前测试场景的流水线测试任务自动化检测目标替换变量,并根据所述目标替换变量自动化替换解析得到的字段中的目标占位符；

验证必填字段、字段类型、芯片专用字段以及目标依赖关系的规范性。

6. 根据权利要求1所述的方法,其特征在于,所述根据所述芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流水线测试任务,包括:

对各所述测试场景的流水线测试任务创建并维护独立测试工作空间；

在各所述测试工作空间内根据所述芯片测试配置字段执行八阶段测试环境重建流程,得到各所述流水线测试任务对应的重构测试环境；

在所述重构测试环境中根据目标测试命令和目标测试用例自动化执行对应的流水线测试任务；

其中,所述目标测试命令和所述目标测试用例基于所述芯片测试配置字段获取。

7. 根据权利要求6所述的方法,其特征在于,所述在各所述测试工作空间内根据所述芯片测试配置字段执行八阶段测试环境重建流程,包括:

根据第一芯片测试配置字段下载核心转储文件并保存至对应测试工作空间的工作目录；

根据第二芯片测试配置字段读取软件包列表,并按顺序依次下载各软件包；

根据第三芯片测试配置字段读取系统环境检查命令列表,并按顺序依次执行每个系统环境检查命令；

根据第四芯片测试配置字段读取并执行芯片检测命令列表,解析芯片检测命令输出值,提取目标关键信息；

根据第五芯片测试配置字段读取环境设置命令列表,并按顺序依次执行每个环境设置命令,以更新宿主机环境变量并创建目录；

根据第六芯片测试配置字段读取并执行驱动安装命令,以安装芯片的内核驱动模块；

根据第七芯片测试配置字段读取并执行容器启动命令列表中的各个容器启动命令,并存储记录容器ID信息；

根据第八芯片测试配置字段读取容器内环境设置命令,对各所述容器内环境设置命令添加容器执行前缀后,按顺序执行各所述容器内环境设置命令。

8. 根据权利要求6所述的方法,其特征在于,所述在所述重构测试环境中根据目标测试命令和目标测试用例自动化执行对应的流水线测试任务,包括:

从所述芯片测试配置字段中筛选芯片测试关联字段,并确定用例操作方法；

根据所述流水线测试任务的任务类型从所述芯片测试关联字段中筛选所述目标测试命令,并从所述用例操作方法中筛选所述目标测试用例；

采用所述目标测试命令根据所述目标测试用例自动化执行对应的流水线测试任务；

在自动化执行对应的流水线测试任务的过程中,存储管理测试执行日志；

其中,所述用例操作方法包括运行测试用例方法、进入容器交互模式方法、显示命令序列方法以及显示崩溃堆栈方法。

9. 根据权利要求1所述的方法,其特征在于,还包括:

获取各所述测试场景的流水线测试任务的任务实时运行状态信息；其中,所述任务实

时运行状态信息包括配置文件路径、容器名称以及环境构建各阶段的完成信息；

所述实时运行状态信息用于在自动化执行各所述测试场景的流水线测试任务的过程中对已完成阶段进行跳过处理。

10. 一种芯片测试装置,其特征在于,包括:

芯片测试配置文件获取模块,用于获取标准化的芯片测试配置文件;其中,所述标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构;

芯片测试配置字段解析模块,用于对所述标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段;

流水线测试任务执行模块,用于根据所述芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流水线测试任务;

所述芯片测试配置字段解析模块还用于:对所述标准化的芯片测试配置文件的各数据结构进行解析和验证,得到初始化芯片测试配置字段;从所述初始化芯片测试配置字段提取芯片领域专用字段;对所述芯片领域专用字段进行结构化配置,得到所述芯片测试配置字段。

11. 一种电子设备,其特征在于,所述电子设备包括:

至少一个处理器;以及

与所述至少一个处理器通信连接的存储器;其中,

所述存储器存储有被所述至少一个处理器执行的计算机程序,所述计算机程序被所述至少一个处理器执行,以使所述至少一个处理器能够执行权利要求1-9中任一所述的芯片测试方法。

12. 一种计算机可读存储介质,其特征在于,所述计算机可读存储介质存储有计算机指令,所述计算机指令用于使处理器执行时实现权利要求1-9中任一所述的芯片测试方法。

13. 一种计算机程序产品,其特征在于,包括计算机程序/指令,其中,所述计算机程序/指令被处理器执行时实现权利要求1-9中任一所述的芯片测试方法。

一种芯片测试方法、装置、电子设备、存储介质及程序

技术领域

[0001] 本发明实施例涉及芯片和测试技术领域,尤其涉及一种芯片测试方法、装置、电子设备、存储介质及程序。

背景技术

[0002] 芯片公司的软件栈由于其本身的复杂度较高,其测试流水线也相应地具有更高的复杂度。

[0003] 目前对芯片进行测试时,无论是普通的芯片还是AI(Artificial Intelligence, 人工智能)芯片,业界普遍采用CI(Continuous Integration,持续集成)/CD(Continuous Delivery,持续交付)流水线进行自动化测试。这种CI/CD流水线的自动化测试方法采用通用配置文件,如JSON(JavaScript Object Notation,JavaScript 对象表示法)或YAML(YAML Ain't Markup Language,一种轻量级且以数据为中心的序列化语言)等格式的通用配置文件,完成测试流程定义与参数配置。

[0004] 然而,在芯片测试中,测试人员往往会根据芯片的具体测试需求提供芯片测试专用的配置文件。芯片测试专用配置文件中可能包括芯片特有字段等专用信息。目前的通用测试流水线缺乏对芯片领域配置的理解能力,其采用的通用配置解析器无法识别芯片测试专用的配置文件中芯片特有字段的含义,从而难以实现芯片测试场景的精确复现。

发明内容

[0005] 本发明实施例提供一种芯片测试方法、装置、电子设备、存储介质及程序,能够快速且精确地复现芯片的专用测试场景。

[0006] 根据本发明的一方面,提供了一种芯片测试方法,包括:

获取标准化的芯片测试配置文件;其中,所述标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构;

对所述标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段;

根据所述芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流水线测试任务。

[0007] 根据本发明的另一方面,提供了一种芯片测试装置,包括:

芯片测试配置文件获取模块,用于获取标准化的芯片测试配置文件;其中,所述标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构;

芯片测试配置字段解析模块,用于对所述标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段;

流水线测试任务执行模块,用于根据所述芯片测试配置字段采用分阶段的方式自

动化执行各测试场景的流水线测试任务。

[0008] 根据本发明的另一方面,提供了一种电子设备,所述电子设备包括:
至少一个处理器;以及与所述至少一个处理器通信连接的存储器;其中,
所述存储器存储有可被所述至少一个处理器执行的计算机程序,所述计算机程序被所述至少一个处理器执行,以使所述至少一个处理器能够执行本发明任一实施例所述的芯片测试方法。

[0009] 根据本发明的另一方面,提供了一种计算机可读存储介质,所述计算机可读存储介质存储有计算机指令,所述计算机指令用于使处理器执行时实现本发明任一实施例所述的芯片测试方法。

[0010] 根据本发明的另一方面,还提供了一种计算机程序产品,包括计算机程序,所述计算机程序在被处理器执行时实现本发明任一实施例所述的芯片测试方法。

[0011] 本发明实施例通过获取标准化的芯片测试配置文件,该标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构等多种数据结构,从而可以对标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段,进而根据芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流水线测试任务。上述芯片测试方法可以有效解决现有通用流水线测试工具难以有效理解芯片领域配置信息,以致无法复现芯片测试场景的问题,能够快速且精确地复现芯片的专用测试场景。

[0012] 应当理解,本部分所描述的内容并非旨在标识本发明的实施例的关键或重要特征,也不用于限制本发明的范围。本发明的其它特征将通过以下的说明书而变得容易理解。

附图说明

[0013] 为了更清楚地说明本发明实施例中的技术方案,下面将对实施例描述中所需要使用的附图作简单地介绍,显而易见地,下面描述中的附图仅仅是本发明的一些实施例,对于本领域普通技术人员来讲,在不付出创造性劳动的前提下,还可以根据这些附图获得其他的附图。

[0014] 图1是本发明实施例一提供的一种芯片测试方法的流程图;
图2是本发明实施例一提供的一种芯片测试装置的结构示意图;
图3是本发明实施例二提供的一种芯片测试方法的流程图;
图4是本发明实施例三提供的一种芯片测试方法的流程图;
图5是本发明实施例四提供的一种芯片测试方法的流程图;
图6是本发明实施例四提供的一种八阶段测试环境重建流程的示意图;
图7是本发明实施例四提供的一种测试工作场景生命周期状态机的结构示意图;
图8是本发明实施例五提供的一种芯片测试装置的示意图;
图9为本发明实施例六提供的一种电子设备的结构示意图。

具体实施方式

[0015] 为了使本技术领域的人员更好地理解本发明方案,下面将结合本发明实施例中的附图,对本发明实施例中的技术方案进行清楚、完整地描述,显然,所描述的实施例仅仅是

本发明一部分的实施例,而不是全部的实施例。基于本发明中的实施例,本领域普通技术人员在没有做出创造性劳动前提下所获得的所有其他实施例,都应当属于本发明保护的范围。

[0016] 需要说明的是,本发明的说明书和权利要求书及上述附图中的术语“第一”、“第二”、“第三”、“第四”、“第五”、“第六”、“第七”以及“第八”等是用于区别类似的对象,而不必用于描述特定的顺序或先后次序。应该理解这样使用的数据在适当情况下可以互换,以便这里描述的本发明的实施例能够以除了在这里图示或描述的那些以外的顺序实施。此外,术语“包括”和“具有”及其任何变形,意图在于覆盖不排他的包含,例如,包含了一系列步骤或单元的过程、方法、系统、产品或设备不必限于清楚地列出的那些步骤或单元,而是可包括没有清楚地列出的或对于这些过程、方法、产品或设备固有的其它步骤或单元。

[0017] 实施例一

图1是本发明实施例一提供的一种芯片测试方法的流程图,本实施例可适用于对包括特定数据结构的芯片测试配置文件进行自动化解析,以自动化执行芯片各测试场景的流水线测试任务的情况,该方法可以由芯片测试装置来执行,该装置可以由软件和/或硬件的方式来实现,并一般可集成在电子设备中,该电子设备可以是终端设备,也可以是服务器设备,只要能够执行芯片测试方法即可,本发明实施例并不对电子设备的具体设备类型进行限定。相应的,如图1所示,该方法包括如下操作:

S110、获取标准化的芯片测试配置文件;其中,所述标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构。

[0018] 其中,芯片测试配置文件可以是专用于对芯片进行测试的配置文件。可选的,需要生成芯片测试配置文件的芯片可以是AI芯片,也还可以是普通的芯片,本发明实施例并不对芯片测试配置文件所适配的芯片类型进行限定。

[0019] 在本发明实施例中,可以通过基于芯片领域特定配置规范的芯片测试配置解析与执行装置(下文简称芯片测试装置)快速且精准地复现芯片的各类测试场景。首先,芯片测试装置可以获取标准化的芯片测试配置文件。可选的,标准化的芯片测试配置文件可以采用五区结构的方式,对芯片在测试过程中所需的配置信息进行整合。标准化的芯片测试配置文件的五区结构可以包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构。

[0020] 其中,元数据区结构(可简称为元数据区)可以用于存储芯片测试所需的元数据。元数据是描述数据的数据,它不包含数据本身的内容,而是提供关于数据的结构、含义、来源和用途等关键信息,帮助人们和系统理解、管理、发现和使用数据。可选的,元数据区结构可以采用元数据字段存储元数据,元数据字段可以是能够描述元数据的相关字段。

[0021] 其中,硬件环境描述区结构(可简称为软件环境描述区)可以为用于描述芯片驱动、运行时库和容器环境的数据结构。硬件环境描述区结构可以采用芯片硬件环境检测字段存储芯片硬件环境检测数据。芯片硬件环境检测字段可以是能够描述芯片硬件环境检测数据的相关字段。芯片硬件环境检测数据主要指在芯片运行或测试过程中,对其所处物理环境及自身工作状态的实时监测参数,用于保障稳定性、可靠性与安全性。这些数据通常由片上传感器或外部监控系统采集,涵盖温度、电压、功耗、湿度和振动等多个维度。

[0022] 其中,软件环境描述区结构(可简称为软件环境描述区)可以为用于描述芯片驱动、运行时库和容器环境的数据结构。软件环境描述区结构可以采用芯片软件环境字段存储芯片软件环境数据。芯片软件环境字段可以是能够描述芯片软件环境数据的相关字段。芯片软件环境数据是指在芯片运行、测试或开发过程中,与其软件系统密切相关的配置、状态和行为信息,用于确保软件与硬件协同工作正常,并支撑功能验证、性能调优与故障诊断。

[0023] 其中,环境配置区结构(可简称为环境配置区)可以为用于描述宿主机和容器内的环境设置的数据结构。环境配置区结构可以采用环境配置字段存储环境配置数据。环境配置字段可以为用于描述环境配置数据的相关字段。环境配置数据是确保芯片在开发、测试与运行阶段能够稳定工作的关键参数集合,涵盖硬件、软件及物理环境的设定信息,直接影响系统兼容性、性能表现与可靠性。

[0024] 其中,测试执行描述区结构(可简称为测试执行描述区)可以为用于描述测试命令和结果数据的数据结构。测试执行描述区结构可以采用测试执行字段存储测试执行数据。测试执行字段可以是用于描述测试执行数据的相关字段。测试执行数据是软件或硬件在测试过程中产生的所有原始记录和结果信息,用于评估系统功能、性能与稳定性,是质量保障和问题追溯的核心依据。

[0025] 在此基础上,为了适应不同类型芯片的测试需求标准化的芯片测试配置文件的数据结构还可以适应扩展。如增设依赖描述区结构(简称为依赖描述区,dependencies)等。示例性的,依赖描述区结构可以配置的字段包括但不限于framework、framework_version、model_artifacts以及dataset_paths等。其中,framework字段可以用于指定开发框架或系统框架的类型,framework_version可以用于指定开发框架或系统框架的版本号,model_artifacts为适用于AI芯片类型的字段,用于配置模型训练、编译或部署过程中生成的所有关键文件和资源。dataset_paths字段指的是指向数据集文件或文件夹的路径。

[0026] 由此可见,芯片测试配置文件在五区结构的设计确保了芯片测试场景的完整性,其中元数据区的配置信息可以提供场景标识和追溯能力,硬件环境描述区的配置信息可以保证硬件依赖可验证,软件环境描述区的配置信息可以保证软件版本可复现,环境配置区的配置信息可以保证环境变量和设置可恢复,测试执行描述区的配置信息可以保证测试用例可重新执行。

[0027] S120、对所述标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段。

[0028] 其中,芯片测试配置字段可以是芯片领域的专用字段。

[0029] 图2是本发明实施例一提供的一种芯片测试装置的结构示意图。在一个具体的例子中,如图2所示,芯片测试装置可以采用三层架构,从上到下分为用户接口层、核心处理层和基础设施层。其中,用户接口层本质为CLI(Command Line Interface,命令行界面)命令行接口,可以提供多种标准化的命令,可覆盖配置解析、环境构建、测试执行以及场景管理的完整生命周期。可选的,用户接口层可以对用户提供自动设置测试环境的命令、一键运行测试用例的命令、进入容器内进行交互式调试的命令、列出所有工作场景的命令、选择并切换到指定工作场景的命令、显示当前场景的详细信息命令、清理工作场景的命令以及重置芯片设备的命令等。

[0030] 示例性的,自动设置测试环境的命令可以为efdbg setup <config_file>,该命令可根据标准化的芯片测试配置文件config_file自动设置测试环境,从而实现核心复现功能。efdbg setup <config_file>通过读取芯片测试配置文件config_file,解析芯片测试配置文件中元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构等五区数据结构的字段,自动执行八阶段环境构建流程。

[0031] 示例性的,一键运行测试用例的命令可以为efdbg run <config_file>,该命令集成了setup(设置)和测试执行的完整流程,能够自动进入容器并执行芯片测试配置文件中的tests(测试)字段定义的测试命令。

[0032] 示例性的,进入容器内进行交互式调试的命令可以为efdbg enter,该命令允许工程师手动进入容器,执行任意调试命令,支持断点调试、日志查看以及文件检查等操作。

[0033] 示例性的,列出所有工作场景的命令可以为efdbg list,该命令可以用于显示当前系统中所有已创建的工作场景,每个工作场景对应一个jobid(工作空间ID),每个工作场景即为一个芯片测试场景。该命令还可以显示测试场景的状态,如CREATED(创建)、RUNNING(运行中)以及PAUSED(已暂停)等。

[0034] 示例性的,选择并切换到指定工作场景的命令可以为efdbg select <jobid>,该命令能够实现秒级测试场景切换,无需重新构建环境,支持多测试场景的并行管理。

[0035] 示例性的,显示当前场景的详细信息命令可以为efdbg show命令,可以明显可以细分为两种子命令,一种子命令可以显示场景执行的完整命令序列,另外一种子命令可以在确定有核心转储文件的情况下,分析测试场景的崩溃堆栈。

[0036] 示例性的,清理工作场景的命令可以为efdbg clean,该命令可以删除指定场景的工作目录和容器,支持批量清理和自动清理策略。

[0037] 示例性的,重置芯片设备的命令可以为efdbg reset,该命令可以重置芯片设备,用于设备状态异常时的恢复。

[0038] 由此可见,用户接口层通过采用统一入口命令efdbg加子命令的形式,为用户提供各种测试功能入口。其中,子命令与核心处理层模块的对应关系为:setup/run子命令用于触发配置解析与环境复现、enter子命令用于进入复现后的容器、list/select/show/clean子命令用于管理复现场景、reset子命令用于控制芯片设备。

[0039] 芯片测试装置的核心处理层可以配置有多种功能模块,各模块各司其职且协同工作。核心处理层的其中一个比较重要的功能模块为配置解析模块。相应的,芯片测试装置可以通过配置解析模块获取标准化的芯片测试配置文件。在通过配置解析模块获取到标准化的芯片测试配置文件之后,可以对芯片测试配置文件的元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构等各数据结构进行解析,得到可以应用于芯片测试领域专用的芯片测试配置字段。可选的,芯片测试配置文件的每个类型的数据结构都可以解析得到部分芯片测试配置字段。

[0040] 芯片测试装置的基础设施层可以提供三大基础能力。示例性的,基础设施层可以通过Docker(一个开源的容器化平台)容器,主要用户通过容器实现测试场景的隔离,芯片测试配置文件中的每个jobid字段可以对应一个独立的容器实例。基础设施层可以通过文件系统(缓存目录)存储工作空间数据。基础设施层还可以通过网络下载下载软件包以及核心转储文件等。也即,基础设施层主要为芯片测试装置提供容器运行、缓存存储与软件包下

载等支撑功能。

[0041] 综上,芯片测试装置通过三层分离设计可以实现接口、逻辑和基础设施的解耦。核心处理层的各个模块通过工作目录和配置文件协同,职责清晰。用户接口层提供统一的CLI命令,从而隐藏内部复杂性,便于用户操作。

[0042] S130、根据所述芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流水线测试任务。

[0043] 其中,流水线测试任务可以指芯片测试通用流水线对芯片进行测试时所执行的测试任务。芯片测试通用流水线可以是目前对芯片进行测试的通用流水线,例如可以是芯片的CI/CD流水线等,本发明实施例并不对芯片测试通用流水线的类型和测试流程进行限定。

[0044] 可选的,芯片测试装置的核心处理层还可以配置有分阶段环境构建模块和测试执行模块。其中,分阶段环境构建模块可以根据对芯片测试配置文件解析的芯片测试配置字段,按照芯片测试场景的复现要求,执行分阶段的环境重建流程。在分阶段环境构建模块执行分阶段的环境重建流程中,可以支持断点续传功能,即通过检查点机制,自动跳过已完成的阶段,从而提高环境重建效率。相应的,测试场景的测试环境重建完成后,芯片测试装置的核心处理层可以结合基础设施层,通过测试执行模块执行芯片测试配置字段中相关测试字段定义的测试命令,复现芯片测试通用流水线中的测试用例,以自动化执行对应测试场景的流水线测试任务。可选的,测试执行模块可以支持测试用例的过滤和覆盖,以提高测试效率。

[0045] 本发明实施例通过获取标准化的芯片测试配置文件,该标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构等多种数据结构,从而可以对标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段,进而根据芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流水线测试任务。上述芯片测试方法可以有效解决现有通用流水线测试工具难以有效理解芯片领域配置信息,以致无法复现芯片测试场景的问题,能够快速且精确地复现芯片的专用测试场景。

[0046] 实施例二

图3是本发明实施例二提供的一种芯片测试方法的流程图,本实施例以上述实施例为基础进行具体化,在本实施例中,给出了获取标准化的芯片测试配置文件的多种具体可选的实现方式。相应的,如图3所示,本实施例的方法可以包括:

S310、采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息。

[0047] 其中,实时状态关联信息可以是在芯片测试通用流水线的各测试阶段实时采集的,多维度的状态信息。

[0048] 在本发明实施例中,可以通过配置实例生成工具,基于芯片测试通用流水线采集的信息自动化生成芯片专用的测试配置文件。其中,配置实例生成工具可以包括状态捕获模块和配置文件生成模块。状态捕获模块和配置文件生成模块之间保持通信连接。状态捕获模块可以负责采集芯片测试通用流水线在芯片测试过程中每个相关测试阶段的状态信息,并将采集的各测试阶段的状态信息进行整合,从而得到实时状态关联信息。示例性的,状态捕获模块可以在CI/CD流水线的环境准备、驱动安装、软件配置以及测试执行等各阶段

实时采集状态信息。状态捕获模块可以将采集的实时状态关联信息存储在内部数据结构中,该内部数据结构例如可以为StateCollector(状态收集器)对象,并在流水线执行完成后统一传递给配置文件生成模块。进一步的,流水线执行完成后,状态捕获模块将实时状态关联信息发送至配置文件生成模块。配置文件生成模块可以对状态捕获模块发送的实时状态关联信息进行解析和整合,从而生成最终标准化的芯片测试配置文件。

[0049] 由于芯片测试通用流水线可能包括多个测试阶段,且每个测试阶段采集的状态信息可能包括多个类型。因此,状态捕获模块可以结合芯片测试需求,对实时状态关联信息进行分类。相应的,配置文件生成模块可以对每个类型的实时状态关联信息提取关键信息,并将提取的关键信息按照标准要求整理生成结构化的配置文件,从而得到标准化的芯片测试配置文件。相对于传统方法需要人工编写配置文件容易遗漏关键信息或产生不一致的技术问题,配置实例生成工具通过状态捕获模块和配置文件生成模块的相互配合,实现了从流水线执行到配置文件生成的自动转换,能够确保配置信息的完整性、准确性和一致性。

[0050] S320、根据所述实时状态关联信息填充所述芯片测试配置文件的预设数据结构,得到初始芯片测试配置文件。

[0051] 其中,预设数据结构可以是考虑芯片特性,针对芯片测试配置文件所特定配置的专用数据结构。本发明实施例中,预设数据结构可以包括但不限于元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构等各类数据结构。初始芯片测试配置文件可以是基于实时状态关联信息和芯片测试配置文件的预设数据结构所初步生成的芯片测试配置文件。

[0052] 在本发明实施例中,配置文件生成模块接收到状态捕获模块采集的全量的实时状态关联信息,并根据实时状态关联信息生成标准化的芯片测试配置文件时,可以通过模板填充引擎、模式验证器和文件序列化器三个组件协作生成标准化的芯片测试配置文件。相应的,配置文件生成模块可以通过模板填充引擎对接收的实时状态关联信息填充到芯片测试配置文件的预设数据结构中,得到初始芯片测试配置文件。

[0053] 在本发明的一个可选实施例中,所述采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息,可以包括:在所述芯片测试通用流水线的环境准备阶段,采集芯片设备硬件信息;在所述芯片测试通用流水线的驱动安装阶段,采集芯片软件版本关联信息;在所述芯片测试通用流水线的软件配置阶段,采集环境变量快照信息;在所述芯片测试通用流水线的测试执行阶段,采集测试命令执行记录信息。

[0054] 其中,芯片测试通用流水线的环境准备阶段是确保测试过程高精度及高可靠性的基础环节,主要包含测试环境、设备、样品和方案的全面就绪准备工作。芯片测试通用流水线的驱动安装阶段是确保自动化测试设备与主机系统之间稳定通信的关键环节。该阶段主要涉及调试器/编程器驱动、USB(Universal Serial Bus,通用串行总线)转串口驱动以及芯片厂商专用驱动的正确部署。芯片软件版本关联信息可以为芯片的软件版本所相关联的信息,例如可以包括但不限于驱动版本号、驱动安装命令及驱动参数等。芯片测试通用流水线的软件配置阶段主要涉及测试程序、测试环境、自动化脚本及工具链的初始化与参数化设置,目的是确保测试流程能够高效且准确地执行芯片功能与性能验证。芯片测试通用流水线的测试执行阶段主要包括晶圆测试、封装测试和系统级测试等,以验证芯片的功能、性能和可靠性。测试命令执行记录信息可以包括但不限于测试命令序列以及核心转储文件路

径等与测试命令的执行情况相关的信息。

[0055] 可选的,状态捕获模块可以包括硬件信息采集器、软件版本记录器、环境变量快照器以及命令执行记录器等多种类型的数据采集器。其中,硬件信息采集器可以用于采集芯片的硬件信息,如GCU (General Computing Unit,通用计算单元) 设备ID、数量以及运行模式等。软件版本记录器可以用于记录驱动版本、运行时版本以及容器镜像版本等信息。环境变量快照器可以用于捕获宿主机和容器内的关键环境变量等信息。命令执行记录器可以用于记录各阶段执行的命令序列,如芯片检测命令:target_check_cmd和驱动安装命令:kmd_install_cmd等。

[0056] 具体的,硬件信息采集器可以在芯片测试通用流水线的环境准备阶段采集芯片设备硬件信息,如通过lspci (List PCI devices,列出PCI设备) 命令检测GCU设备ID和数量等。可选的,在环境准备阶段,状态捕获模块还可以通过环境变量快照器来捕获宿主机环境变量,并通过命令执行记录器来记录系统检查命令如system_check_cmd和芯片检测命令如target_check_cmd。相应的,状态捕获模块可以在芯片测试通用流水线的驱动安装阶段,通过软件版本记录器和命令执行记录器采集存储芯片软件版本关联信息。例如,状态捕获模块可以通过软件版本记录器记录驱动版本信息,并通过命令执行记录器记录驱动安装命令如kmd_install_cmd及该驱动安装命令包括的驱动安装参数等。

[0057] 相应的,状态捕获模块可以在芯片测试通用流水线的软件配置阶段,通过软件版本记录器和命令执行记录器采集存储芯片的环境变量快照信息。可选的,环境变量快照信息可以包括但不限于容器镜像版本、容器启动参数、SDK (Software Development Kit,软件开发工具包) 版本以及环境变量设置命令等。例如,状态捕获模块可以通过软件版本记录器记录芯片的容器镜像版本和SDK版本等信息,并通过命令执行记录器记录容器启动命令如docker_launch_cmd和容器内环境设置命令等,如通过SDK安装时,容器内环境设置命令可以为container_env_setup。相应的,状态捕获模块可以在芯片测试通用流水线的测试执行阶段,通过命令执行记录器记录存储芯片的各项测试命令信息。示例性的,状态捕获模块可以通过命令执行记录器记录存储测试命令前缀以及各种类型的测试命令构成的测试命令序列。如果芯片在芯片测试通用流水线的测试执行阶段发生崩溃,还可以记录用于崩溃分析的核心转储文件路径。核心转储文件路径是指测试流程在进程异常终止时,将测试状态保存到磁盘的指定位置。

[0058] 由此可见,状态捕获模块通过各类数据采集器在芯片测试通用流水线的各阶段互相协作,以采集芯片测试配置文件所需的全量状态信息。

[0059] 在本发明的一个可选实施例中,所述根据所述实时状态关联信息填充所述芯片测试配置文件的预设数据结构,可以包括:从所述实时状态关联信息中提取元数据字段,并根据所述元数据字段填充所述芯片测试配置文件的元数据区结构;从所述实时状态关联信息中提取芯片硬件环境检测字段,并根据所述芯片硬件环境检测字段填充所述芯片测试配置文件的硬件环境描述区结构;从所述实时状态关联信息中提取芯片软件环境字段,并根据所述芯片软件环境字段填充所述芯片测试配置文件的软件环境描述区结构;从所述实时状态关联信息中提取环境配置字段,并根据所述环境配置字段填充所述芯片测试配置文件的环境配置区结构;从所述实时状态关联信息中提取测试执行字段,并根据所述测试执行字段填充所述芯片测试配置文件的测试执行描述区结构。

[0060] 可选的,预设数据结构可以为按芯片测试配置规范所配置的五区结构组织。模板填充引擎可以将实时状态关联信息按照芯片测试配置规范所配置的五区数据结构进行数据填充操作。具体的,模板填充引擎可以从实时状态关联信息中提取元数据字段,并根据元数据字段填充芯片测试配置文件的元数据区结构。示例性的,元数据字段可以包括但不限于version(版本号)、jobid(工作空间ID)、repo(配置项)、job_url(工作空间链接)以及cdash_url等。其中,version字段用于记录配置规范的版本号,用于向后兼容性管理,当规范升级时通过版本号识别;jobid字段为测试任务的唯一标识符,支持变量替换,用于区分不同的测试任务;repo字段为关联的代码仓库地址,用于追溯测试场景对应的源代码版本;job_url字段为关联的芯片测试通用流水线任务的URL(Uniform Resource Locator,统一资源定位符),用于追溯芯片测试配置文件的生成来源;cdash_url字段为测试结果仪表板URL,用于查看测试执行结果和历史记录。

[0061] 具体的,模板填充引擎还可以从实时状态关联信息中提取芯片硬件环境检测字段,并根据芯片硬件环境检测字段填充芯片测试配置文件的硬件环境描述区结构。硬件环境描述区结构的描述数据为区别于通用测试配置文件的核心差异化字段,可以体现芯片测试的特殊性。示例性的,芯片硬件环境检测字段可以包括但不限于target_check_cmd字段和system_check_cmd字段。其中,target_check_cmd字段可以包括芯片检测命令列表,用于验证目标芯片(如目标AI加速卡或通用芯片等)是否存在及其状态,例如可以通过lspci命令检测GCU设备的厂商ID和设备ID。system_check_cmd字段可以包括系统环境检查命令列表,用于验证操作系统挂载点、内核版本等系统级依赖是否满足要求。

[0062] 具体的,模板填充引擎还可以从实时状态关联信息中提取芯片软件环境字段,并根据芯片软件环境字段填充芯片测试配置文件的软件环境描述区结构。示例性的,芯片软件环境字段可以包括但不限于image(容器镜像)、container(容器实例)、package_url(软件包URL)、kmd_install_cmd(驱动安装命令)、docker_launch_cmd(容器启动命令)、container_env_prefix(从外部资源注入到容器的环境变量添加统一的前缀)以及download_cmds(软件包下载命令)等字段。其中,image字段可以包括容器镜像的完整地址(含仓库路径和版本标签);container字段可以包括容器实例名称,支持变量替换以实现多场景隔离;package_url字段可以包括软件包(如SDK和工具链等)的下载地址;kmd_install_cmd字段可以包括芯片内核驱动模块(Kernel Mode Driver,KMD)的安装命令,安装命令包含驱动安装参数如--driver -y --no-auto-load等;docker_launch_cmd字段可以包括容器启动命令列表,容器启动命令包含挂载路径和设备映射等参数;container_env_prefix字段可以包括容器内命令执行前缀,如docker exec -it {container},用于后续命令的拼接;download_cmds字段可以包括软件包下载命令序列,软件包下载命令序列可按顺序执行以获取所需文件。

[0063] 具体的,模板填充引擎还可以从实时状态关联信息中提取环境配置字段,并根据环境配置字段填充芯片测试配置文件的环境配置区结构。示例性的,环境配置字段可以包括但不限于host_env_setup(宿主机环境设置)字段、container_env_setup(容器内环境设置)字段以及test_env_type(测试环境类型标识)字段。其中,host_env_setup字段可以包括宿主机环境设置命令列表,如设置环境变量和创建目录等;container_env_setup字段可以包括容器内环境设置命令列表,如安装SDK和配置库路径等;test_env_type字段可以包

括测试环境类型标识,如dev(development,开发环境)、CI(Continuous Integration,持续集成)和prod(生产环境)等,用于区分不同环境的配置策略。

[0064] 具体的,模板填充引擎还可以从实时状态关联信息中提取测试执行字段,并根据测试执行字段填充芯片测试配置文件的测试执行描述区结构。示例性的,测试执行字段可以包括但不限于test_prefix(测试命令前缀)、tests(测试命令序列)以及corefile(核心转储文件路径)等字段。其中,test_prefix字段可以包括测试命令的公共前缀,测试命令如可以为切换目录以及激活环境等;tests字段可以包括测试命令序列,可按顺序执行各个测试用例;corefile字段可以包括核心转储文件的路径,用于崩溃问题的调试分析。

[0065] 当对预设数据结构的五区结构填充完成后,可以生成dict(一种字典数据结构,本质为键值对集合)结构格式的初始芯片测试配置文件。

[0066] 上述预设数据结构的五区结构设计可以直观反映芯片测试环境的依赖层次即元数据→硬件→软件→环境→执行,每层职责清晰,便于自动化解析。芯片测试配置文件中的target_check_cmd和kmd_install_cmd等字段是针对AI芯片测试场景的专用抽象,填补了通用配置格式的空白。同时,芯片测试配置文件支持{user}和{version}等占位符的自动变量替换机制,可以实现芯片配置模板化和复用。芯片测试配置文件的所有执行类字段均为列表类型,明确了命令的执行顺序和依赖关系。

[0067] S330、对所述初始芯片测试配置文件中的配置数据进行规范化验证。

[0068] 为了进一步保证芯片测试配置文件的可用性和准确性,在生成初始芯片测试配置文件之后,可以进一步对初始芯片测试配置文件中的配置数据进行规范化验证,以验证初始芯片测试配置文件内的配置数据是否符合芯片测试配置的规范。

[0069] 在本发明的一个可选实施例中,所述对所述初始芯片测试配置文件中的配置数据进行规范化验证,可以包括:对所述初始芯片测试配置文件中的必填字段的完整性进行验证;对所述初始芯片测试配置文件中的字段类型的正确性进行验证;对所述初始芯片测试配置文件中的芯片专用字段的有效性进行验证;对所述初始芯片测试配置文件中的测试场景必要字段的覆盖程度进行验证。

[0070] 具体的,配置文件生成模块可以通过模式验证器对初始芯片测试配置文件中的配置数据进行规范化验证,并检查字段完整性和数据类型等。模式验证器可以验证初始芯片测试配置文件中的必填字段的完整性。可选的,必填字段可以包括但不限于meta.version(元数据结构版本)、meta.jobid(元数据所属工作空间)以及software.image(记录包含运行时库的容器镜像的字段)等。同时,模式验证器可以验证初始芯片测试配置文件中的字段类型的正确性。例如,需要验证tests字段类型必须是list,kmd_install_cmd字段类型必须是string等。模式验证器还可以验证初始芯片测试配置文件中的芯片专用字段的有效性,如验证target_check_cmd字段是否包含AI芯片检测命令等。最后,模式验证器还可以验证初始芯片测试配置文件中的测试场景必要字段的覆盖程度。可选的,测试场景可以包括但不限于核心转储场景、设备测试场景以及库API(Application Programming Interface,应用程序编程接口)测试场景等。相应的,测试场景必要字段可以是上述各类测试场景所必须配置的字段。

[0071] 相应的,如果初始芯片测试配置文件通过验证,则可以继续后续流程;如果初始芯片测试配置文件未通过验证,则可以根据验证过程中检查出的错误项自动化生成错误报

告。

[0072] S340、在确定所述初始芯片测试配置文件通过验证的情况下,将所述初始芯片测试配置文件进行序列化处理,得到所述标准化的芯片测试配置文件。

[0073] 如果确定初始芯片测试配置文件通过验证,配置文件生成模块可以通过文件序列化器进一步对初始芯片测试配置文件进行序列化处理,如将初始芯片测试配置文件的配置数据序列化为JSON和/或YAML格式。在本发明的一个可选实施例中,在文件序列化器的序列化过程中,可以对初始芯片测试配置文件按照设定标准格式进行格式化处理和缩进处理,以提高可读性;还可以对初始芯片测试配置文件添加配置文件生成时间戳和生成系统标识等。文件序列化器对初始芯片测试配置文件经过序列化处理,可以输出标准化的且可读性强的自描述配置文件,从而得到标准化的芯片测试配置文件。

[0074] 可选的,生成的标准化的芯片测试配置文件作为芯片测试通用流水线的输出产物,可以存档到构建产物仓库(Artifact Repository),也还可以通过job_url等字段关联到芯片测试通用流水线的测试任务以对目标芯片进行测试,或者也还可以供给配套装置读取并用于芯片测试环境的快速复现等。本发明实施例并不对标准化的芯片测试配置文件的具体应用场景进行限制。

[0075] S350、对所述标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段。

[0076] S360、根据所述芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流水线测试任务。

[0077] 上述技术方案针对现有测试通用流水线缺乏专门针对芯片测试场景的标准化配置描述语言,导致芯片测试环境难以精确描述和复现的问题,提供了一种专用于描述芯片测试场景的领域特定测试配置文件的生成方法,该方法通过采用分层分区结构对片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息进行填充整理,将芯片测试环境的完整描述信息组织为五个功能区域,每个区域包含若干字段,能够完整、精确地描述芯片软件栈测试所需的硬件环境、软件环境、依赖关系和执行配置,实现芯片测试领域字段约束的结构化规范和芯片测试复现问题领域知识的系统性抽象,能够提高芯片测试配置文件中配置信息的完整性、精确性和标准化程度,有利于实现芯片测试场景的标准化表达和精确复现。

[0078] 实施例三

图4是本发明实施例三提供的一种芯片测试方法的流程图,本实施例以上述实施例为基础进行具体化,在本实施例中,给出了对标准化的芯片测试配置文件的各数据结构进行解析的多种具体可选的实现方式。相应的,如图4所示,本实施例的方法可以包括:

S410、获取标准化的芯片测试配置文件。

[0079] 其中,所述标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构。

[0080] 可以理解的是,生成的标准化的芯片测试配置文件需要被解析和执行才能实现芯片专用测试场景的复现。而目前的通用配置解析器无法识别标准化的芯片测试配置文件中诸如target_check_cmd和kmd_install_cmd等芯片特有字段的含义,无法进行芯片测试领域特定的字段验证过程,如通过lspci检测AI加速卡是否存在等,且通用配置解析器缺乏对

芯片硬件检测-驱动安装-容器部署依赖关系等各测试阶段和相关配置数据的理解能力。为此,本发明实施例提供的芯片测试装置针对芯片领域专用测试数据强化了芯片测试配置信息的解析能力。

[0081] 具体的,芯片测试装置可以通过配置解析模块采用多种可选的方式获取标准化的芯片测试配置文件。配置解析模块的核心能力是理解芯片测试场景的字段约束和变量替换,其可以包括:输入接口、内部组件和输出接口三个部分。可选的,配置解析模块的输入接口支持三种芯片测试配置文件的输入方式,并通过内部组件中的配置加载器具体获取芯片测试配置文件。例如,配置加载器可以通过receipt_path字段(指存储收据文件的本地或服务器路径的自定义字段)配置本地芯片测试配置文件的路径,如“/path/to/receipt.json”,并通过load_from_file(path)方法(自定义的用于从指定文件路径加载数据的自定义或上下文相关方法)从本地文件系统读取芯片测试配置文件,适用于测试工程师已经下载了芯片测试配置文件到本地,或芯片测试配置文件由脚本自动生成的使用场景。当芯片测试配置文件不存在时配置加载器可抛出异常,格式错误时配置加载器可报告语法问题。

[0082] 或者,配置加载器还可以通过receipt_url字段(通常指一个指向收据的URL地址的字段,用于程序化访问或验证该收据内容)远程配置芯片测试配置文件的URL,并通过load_from_url(url)方法(自定义的用于从指定URL下载并加载资源的辅助函数)从远程服务器下载芯片测试配置文件,适用于芯片测试配置文件存储在构建产物仓库,可通过URL直接获取的使用场景。当出现网络错误时配置加载器可重试,下载失败时配置加载器可抛出异常。

[0083] 或者,配置加载器还可以维护一个标签映射表,该标签映射表可以记录芯片测试配置文件与预设标签字符串之间的映射关系。相应的,配置加载器可以通过tag_string字段(通常指某种编码、格式或元数据中用于以字符串形式标识标签tag的字段)指定标签字符串,并通过load_from_tag(tag)方法(自定义的反序列化/加载方法,用于根据标签tag从存储中恢复某个状态或对象)根据指定的标签字符串查询标签映射表得到对应芯片测试配置文件的路径,用户无需记忆完整配置文件路径即可快速定位并下载获取芯片测试配置文件。通过标签字符串快速定位获取芯片测试配置文件的方式无需手动指定配置文件路径,适用于快速复现常用测试场景的使用场景。当制定的标签字符串不存在时,配置加载器可以抛出异常。可选的,在生成标准化的芯片测试配置文件的过程中,针对每一个完成的芯片测试配置文件,都可以按需对应配置标签字符串。

[0084] S420、对所述标准化的芯片测试配置文件的各数据结构进行解析和验证,得到初始化芯片测试配置字段。

[0085] 其中,初始化芯片测试配置字段可以是芯片测试装置对标准化的芯片测试配置文件的各数据结构进行初步解析所得到的字段。

[0086] 配置解析模块的内部组件还可以包括模式验证器、变量替换器以及输出格式化器。配置解析模块的配置解析器可以对获取的芯片测试配置文件的元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构等各数据结构进行解析,得到解析的字段。进一步的,配置解析模块通过模式验证器对芯片测试配置文件的各数据结构解析得到的字段进行验证。在这过程中,配置解析模块还可以通过变量

替换器对解析的字段执行自动的变量替换操作。

[0087] 在本发明的一个可选实施例中,所述对所述标准化的芯片测试配置文件的各数据结构进行解析和验证,可以包括:根据所述标准化的芯片测试配置文件的目标数据结构解析得到芯片检测命令字段以及驱动安装命令字段;根据当前测试场景的流水线测试任务自动化检测目标替换变量,并根据所述目标替换变量自动化替换解析得到的字段中的目标占位符;验证必填字段、字段类型、芯片专用字段以及目标依赖关系的规范性。

[0088] 具体的,配置解析模块通过配置解析器获取芯片测试配置文件后,可以通过配置解析器解析芯片测试配置文件的目标数据结构。例如,配置解析器解析硬件环境描述区结构以获取芯片检测命令字段target_check_cmd,并解析软件环境描述区结构以获取驱动安装命令字段kmd_install_cmd。变量替换器可以负责处理配置文件中的占位符变量。具体的,在测试过程中,变量替换器可以自动检测测试环境中的目标替换变量,并基于检测到的目标替换变量自动化替换解析得到的字段中的目标占位符。示例性的,芯片测试配置字段中的目标占位符可以包括但不限于{user}(表示用户)和{version}(表示版本)等,相应的,测试环境中的目标替换变量例如可以包括但不限于{'user': 'zhangsan', 'version': 'v1.0'}等。可选的,变量替换器可以遍历芯片测试配置文件中的所有字符串类型字段,以查找占位符模式确定目标占位符。如果目标替换变量为空,变量替换器可以使用默认值或抛出警告。示例性的,假设芯片测试配置字段的原始值为:container: "{user}_test_{version}",则变量替换器执行变量替换后更新为:container: "zhangsan_test_v1.0"。

[0089] 具体的,配置解析模块可以通过模式验证器验证芯片测试配置文件的各数据结构以及对应的字段是否符合芯片测试的配置规范。示例性的,模式验证器验证必填字段的规范性时,可以验证meta.version、meta.jobid和software.image等必填字段是否存在;检查字段类型的规范性时,可以验证tests字段是否为列表类型以及kmd_install_cmd是否为字符串类型等。验证芯片专用字段的规范性时可以验证target_check_cmd和kmd_install_cmd等芯片测试领域特定字段的格式是否正确。同时,模式验证器还可以检查依赖关系的规范性。例如,如果模式验证器确定corefile字段非空,则必须提供下载命令等。如果验证失败,模式验证器可以返回详细的错误信息,如缺少哪些字段以及哪些字段类型错误等。验证完成后,模式验证器可以输出包括是否验证通过和/或错误列表信息等验证结果对象。

[0090] 由此可见,模式验证器的验证内容和芯片测试配置文件在生成过程中需要验证的内容有交叉重叠的内容,也有区别性的内容。

[0091] S430、从所述初始化芯片测试配置字段提取芯片领域专用字段。

[0092] S440、对所述芯片领域专用字段进行结构化配置,得到所述芯片测试配置字段。

[0093] 相应的,配置解析模块可以通过输出格式化器对解析获取的初始化芯片测试配置字段进一步进行字段筛选处理,从而从全量的字段中提取芯片领域专用字段如target_check_cmd和kmd_install_cmd等,并将其组织为结构化的专用配置信息。具体的,输出格式化器可以提取芯片测试配置文件的五区结构字段,以根据提取的字段为每个阶段的测试命令生成执行计划,可按八阶段环境构建流程分类,最终基于jobid生成工作目录路径。也即,输出格式化器可以输出内部配置对象,供其他模块如场景管理模块和分阶段环境构建模块等内部使用。

[0094] S450、根据所述芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流

流水线测试任务。

[0095] 配置解析模块的输出接口可以输出原始的芯片测试配置文件以及对其解析生成的芯片测试配置字段的内部表示,还可以输出芯片测试配置字段的验证结果供核心处理层的其他功能模块使用。任何步骤失败时,输出接口都会抛出异常并提供详细错误信息。相应的,核心处理层的其他功能模块可以根据配置解析模块输出的芯片测试配置字段,采用分阶段的方式自动化执行各测试场景的流水线测试任务。

[0096] 在一个具体的例子中,以下为芯片测试装置内核心处理层的配置解析模块的形式化定义:

```
ConfigParserModule {  
  // 输入接口-支持多种输入方式  
  input: {  
    receipt_path: string    // 本地文件路径  
    receipt_url: string     // 远程URL  
    tag_string: string      // 预设标签(如topsinference:3.6.105)  
  }  
  // 内部组件  
  components {  
    // 配置加载器  
    ConfigLoader {  
      load_from_file(path) → dict  
      load_from_url(url) → dict  
      load_from_tag(tag) → dict // 从预设模板加载  
    }  
    // 变量替换器(核心组件)  
    VariableReplacer {  
      // 支持的变量类型  
      supported_variables: {  
        "{user}": getpass.getuser(),  
        "{version}": tag_version,  
        "{topsrider_url}": package_url  
      }  
      replace_variables(config) → dict  
    }  
    // 预设标签管理器  
    TagManager {  
      tags: {  
        "topsinference": {path, description, versions},  
        "vllm": {path, description, versions},  
        "train-ds": {path, description, versions}
```

```
    }
    has_tag(tag) → bool
    get_versions(tag) → list
    get_latest_version(tag) → string
    get_path(tag, version) → string
}
// 字段提取器
FieldExtractor {
    get_jobid(config) → string
    get_container(config) → string
    get_image(config) → string
    get_tests(config) → list
    // ... 其他字段提取方法
}
}
// 输出接口-结构化的配置信息
output: ParsedConfig {
    jobid: string
    container: string
    image: string
    receipts: dict           // 完整的芯片测试配置文件
    work_dir: string         // 工作目录
    release_dir: string      // 发布目录
}
}
```

上述技术方案通过芯片测试装置对芯片测试配置文件进行解析和验证,能够识别并处理芯片测试场景的专用字段,支持本地文件、远程URL两种配置文件的直接输入方式,以及通过预设标签间接定位配置文件的方式,可以提高芯片测试装置的易用性;支持配置文件模板化,通过变量替换实现配置复用;通过模式验证器确保配置文件的完整性和正确性。

[0097] 实施例四

图5是本发明实施例四提供的一种芯片测试方法的流程图,本实施例以上述实施例为基础进行具体化,在本实施例中,给出了自动化执行各测试场景的流水线测试任务的多种具体可选的实现方式。相应的,如图5所示,本实施例的方法可以包括:

S510、获取标准化的芯片测试配置文件。

[0098] 其中,所述标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构。

[0099] S520、对所述标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段。

[0100] S530、对各所述测试场景的流水线测试任务创建并维护独立测试工作空间。

[0101] 目前,芯片测试通用流水线在实现芯片的测试任务时往往存在多测试场景并行管理的困难,排查不同问题时需要切换不同的测试环境,测试环境之间可能存在相互污染,缺乏基于jobid的测试场景隔离机制。

[0102] 为了解决上述技术问题,示例性的,如图2所示,芯片测试装置的核心处理层还可以配置有场景管理模块,用于管理多个测试场景的生命周期,实现测试场景的相互隔离。可选的,测试工作空间可以以jobid为标识,存储在缓存目录中。具体的,场景管理模块可以对各测试场景的流水线测试任务创建并维护独立测试工作空间。一个流水线测试任务可以采用一个jobid标识一个测试工作空间,同时可以管理多测试场景的生命周期,包括测试场景的创建、运行、暂停和销毁等,支持多测试场景的并行和秒级切换。场景管理模块可以采用多种不同类型的数据结构来定义场景管理所需的核心数据结构,并可提供多种类型的场景管理方法。

[0103] 在一个具体的例子中,示例性的,第一种数据结构可以为适配工作场景的数据结构,该数据结构可以用于表示单个测试场景的完整信息。适配工作场景的数据结构的各字段含义可以如下:jobid字段:任务的唯一标识符,来自配置文件的meta.jobid字段,用于区分不同的测试场景,作为工作目录名称的一部分;receipt_path字段:配置文件的本地路径,用于记录该测试场景对应的配置文件位置,用于后续重新加载配置;container字段:关联的Docker容器名称,用于记录该测试场景使用的容器实例名称,来自芯片测试配置文件的software.container字段,在场景切换时,可通过容器名称快速进入对应容器;status字段:场景的当前状态(枚举类型),其可选值包括:CREATED:场景已创建,环境构建完成,但容器未运行;RUNNING:容器正在运行中,可随时进入;PAUSED:容器已暂停(通过docker pause暂停);STOPPED:容器已停止;status字段可以用于判断场景的可用状态,决定是否需要重启容器;created_at字段:场景创建时间戳,用于记录场景的创建时间,还可以用于自动清理过期场景;release_dir字段:场景的工作目录路径,用于存储该场景的配置文件、核心转储以及日志等所有文件。

[0104] 在一个具体的例子中,示例性的,第二种数据结构可以为适配场景注册表的数据结构,该数据结构可以管理全局的测试场景信息,各字段含义如下:cache_dir字段:缓存根目录路径,所有测试场景的工作目录都存储在该根目录下;latest_setup_receipt字段:最近一次setup(设置)命令使用的芯片测试配置文件路径,用于记录当前活跃的测试场景,efdbg enter等无参数命令会进入该场景,该字段会在每次执行efdbg setup或efdbg select命令时更新;cache_dirs字段:历史缓存目录列表,用于记录所有曾经创建的测试场景目录,供efdbg list命令使用,该字段的数据类型为列表,每个元素是一个测试场景的目录路径。

[0105] 在一个具体的例子中,示例性的,第三种数据结构可以为适配场景状态文件的数据结构,该数据结构可以存储在每个场景目录下的job_status.json文件,记录场景的构建状态,各字段含义如下:setup_efdbg字段:场景是否已完成环境构建,其可选值包括“yes”或“no”,用于判断测试场景是否已经完成设置流程,避免重复构建,其更新时机为:efdbg setup命令成功完成后设置为“yes”;receipt_url字段:配置文件的原始URL(如果配置文件来自远程),用于记录芯片测试配置文件的来源,以用于追溯和重新下载;receipt_path字

段:配置文件的本地路径,用于记录配置文件在工作目录中的保存位置;release_dir字段:工作目录的绝对路径,用于记录该测试场景的工作目录位置;container字段:关联的容器名称,用于记录该测试场景使用的容器实例名称。

[0106] 在一个具体的例子中,示例性的,场景管理模块可以提供设置工作场景的方法:cmd_setup,对应efdbg setup <config_file>命令,该方法执行流程如下:

步骤1:加载并解析芯片测试配置文件,调用配置解析模块的parse()方法(通常指将字符串或文本按特定格式解析并转换为对应数据类型或结构的操作)验证芯片测试配置文件的合法性,提取jobid字段作为测试场景标识。

[0107] 步骤2:根据jobid创建工作目录:将芯片测试配置文件复制到工作目录,初始化测试工作空间的状态文件:job_status.json文件,setup_efdbg: "no"。

[0108] 步骤3:构建执行分阶段测试环境:调用分阶段环境构建模块的build()方法(创建方法),按八阶段流程构建环境,八阶段可以是下载核心转储、下载软件包、系统环境检查、芯片硬件检测、宿主机环境设置、驱动安装、容器启动以及容器内环境设置等各个阶段。

[0109] 步骤4:更新测试场景状态:更新job_status.json文件:setup_efdbg: "yes",更新最近使用过的.json场景配置文件:WorkspaceRegistry.latest_setup_receipt为当前配置文件路径,将测试场景添加到全部场景缓存目录:WorkspaceRegistry.cache_dirs列表中,返回值:true(成功)或false(失败)。

[0110] 在一个具体的例子中,示例性的,场景管理模块还可以提供列出所有场景方法:cmd_list,对应efdbg list命令,该方法的执行流程如下:

步骤1:扫描缓存目录:遍历cache_dir下的所有子目录,每个子目录代表一个场景,目录名即jobid。

[0111] 步骤2:读取测试场景状态:读取每个测试场景目录下的job_status.json文件,提取场景的jobid、container、receipt_path和setup_efdbg等信息,检查容器状态。

[0112] 步骤3:返回场景列表:list[WorkspaceInfo],列表中每个WorkspaceInfo元素包含场景的索引、jobid、状态以及创建时间等信息。

[0113] 在一个具体的例子中,示例性的,场景管理模块还可以提供选择工作场景方法:cmd_select,对应efdbg select命令,该方法的执行流程如下:

步骤1:根据索引获取测试场景信息:从cache_dirs列表中获取指定索引的测试场景目录,读取该测试场景目录的job_status.json,提取receipt_path字段。

[0114] 步骤2:更新活跃场景:更新WorkspaceRegistry.latest_setup_receipt为该场景的配置文件路径,后续的efdbg enter和efdbg run等命令将自动使用该测试场景。返回值:true(成功)或false(场景不存在)。

[0115] 在一个具体的例子中,示例性的,场景管理模块还可以提供清理工作场景方法:cmd_clean,对应efdbg clean或efdbg clean命令,该方法的执行流程如下:

步骤1:定位场景:根据测试空间标识workspace_id(即jobid或索引)找到对应的测试场景目录。如果指定“-all”参数,则定位所有测试场景。

[0116] 步骤2:停止并删除容器:执行docker stop {container}命令停止容器,执行docker rm {container}命令删除容器实例。

[0117] 步骤3:删除测试场景的工作目录,从WorkspaceRegistry.cache_dirs中移除该测

试场景。返回值:true (成功)或false (场景不存在)。

[0118] 在一个具体的例子中,示例性的,场景管理模块还可以提供自动清理过期场景方法:auto_clean_up,该方法在efdbg setup命令执行时自动触发,执行流程如下:

步骤1:扫描所有场景:遍历cache_dirs下的所有测试场景目录,读取每个测试场景的created_at时间戳。

[0119] 步骤2:清理过期场景:如果当前时间减去created_at时间的结果大于预设的阈值,则认为测试场景过期,可以调用清理测试场景的cmd_clean()方法清理该测试场景。返回值:清理的场景数量。

[0120] 上述场景管理模块可以提供测试场景隔离功能,每个测试场景拥有独立的工作目录和容器实例,互不干扰;通过更新活跃场景指针,无需重新构建环境,可实现测试场景的秒级切换;通过job_status.json文件持久化场景状态,支持断点续传,状态持久化;通过自动清理可有效避免缓存目录无限增长。

[0121] 在一个具体的例子中,以下是场景管理模块的形式化定义:

```
WorkspaceManagerModule {  
  // 数据结构  
  data_structures {  
    Workspace {  
      jobid: string           // 任务标识(来自配置文件)  
      receipt_path: string    // 配置文件路径  
      container: string       // 关联的容器名称  
      status: enum[CREATED, RUNNING, PAUSED, STOPPED]  
      created_at: timestamp  
      release_dir: string     // 工作目录  
    }  
    WorkspaceRegistry {  
      cache_dir: string       // 缓存根目录(如~/efdbg_cache/)  
      latest_setup_receipt: string // 最近设置的配置文件路径  
      cache_dirs: list        // 历史缓存目录列表  
    }  
    JobStatus {  
      setup_efdbg: string     // 设置状态(yes/no)  
      receipt_url: string     // 配置文件URL  
      receipt_path: string    // 本地配置文件路径  
      release_dir: string     // 发布目录  
      container: string       // 容器名称  
    }  
  }  
  // 操作接口  
  operations {
```

```

// 设置工作场景(对应setup命令)
cmd_setup(receipt: string) → bool {
    1. 加载并解析配置文件
    2. 创建工作目录(cache_dir/jobid/)
    3. 执行分阶段环境构建
    4. 更新JobStatus状态
}
// 列出所有场景(对应list命令)
cmd_list() → list[WorkspaceInfo] {
    1. 扫描缓存目录下的子目录
    2. 读取各目录的job_status.json
    3. 返回场景列表
}
// 选择工作场景(对应select命令)
cmd_select(index: int) → bool {
    1. 根据索引获取配置文件路径
    2. 更新latest_setup_receipt
}
// 清理工作场景(对应clean命令)
cmd_clean(workspace_id: string, all: bool) → bool {
    1. 停止关联的容器
    2. 删除工作目录
    3. 支持清理单个或全部场景
}
// 自动清理过期场景
auto_clean_up(outdate_days: int) → int {
    1. 扫描缓存目录
    2. 删除超过指定天数的场景
}
}
}

```

S540、在各所述测试工作空间内根据所述芯片测试配置字段执行八阶段测试环境重建流程,得到各所述流水线测试任务对应的重构测试环境。

[0122] 其中,重构测试环境可以是重新构建复现的测试环境。

[0123] 目前,芯片测试通用流水线在实现芯片的测试任务时往往存在环境构建过程缺乏芯片特有验证的问题,以致无法执行target_check_cmd命令验证芯片是否存在,无法自动处理kmd_install_cmd中的驱动安装参数,且缺乏分阶段构建和断点续传能力。

[0124] 为了上述技术问题,示例性的,如图2所示,芯片测试装置的核心处理层还可以配置有分阶段环境构建模块,用于采用分阶段流水线设计,根据对芯片测试配置文件解析得

到的芯片测试配置字段,对每个需要执行流水线测试任务的测试工作空间自动化执行八阶段环境重建流程,从而快速复现各流水线测试任务对应的重构测试环境。可选的,八阶段测试环境重建流程可以包括下载核心转储、下载软件包、系统检查、硬件检测、宿主机环境设置、驱动安装、容器启动以及容器内环境设置等各阶段流程。每个阶段的测试环境重建流程可以对应部分芯片测试配置字段。

[0125] 在本发明的一个可选实施例中,所述在各所述测试工作空间内根据所述芯片测试配置字段执行八阶段测试环境重建流程,可以包括:根据第一芯片测试配置字段下载核心转储文件并保存至对应测试工作空间的工作目录;根据第二芯片测试配置字段读取软件包列表,并按顺序依次下载各软件包;根据第三芯片测试配置字段读取系统环境检查命令列表,并按顺序依次执行每个系统环境检查命令;根据第四芯片测试配置字段读取并执行芯片检测命令列表,解析芯片检测命令输出值,提取目标关键信息;根据第五芯片测试配置字段读取环境设置命令列表,并按顺序依次执行每个环境设置命令,以更新宿主机环境变量并创建目录;根据第六芯片测试配置字段读取并执行驱动安装命令,以安装芯片的内核驱动模块;根据第七芯片测试配置字段读取并执行容器启动命令列表中的各个容器启动命令,并存储记录容器ID信息;根据第八芯片测试配置字段读取容器内环境设置命令,对各所述容器内环境设置命令添加容器执行前缀后,按顺序执行各所述容器内环境设置命令。

[0126] 其中,第一芯片测试配置字段可以用于在测试环境重建流程中完成第一阶段即下载核心转储文件阶段的部分测试环境重建操作。第二芯片测试配置字段可以用于在测试环境重建流程中完成第二阶段即下载软件包阶段的部分测试环境重建操作。第三芯片测试配置字段可以用于在测试环境重建流程中完成第三阶段即系统环境检查阶段的部分测试环境重建操作。第四芯片测试配置字段可以用于在测试环境重建流程中完成第四阶段即芯片硬件检测阶段的部分测试环境重建操作。第五芯片测试配置字段可以用于在测试环境重建流程中完成第五阶段即宿主机环境设置阶段的部分测试环境重建操作。第六芯片测试配置字段可以用于在测试环境重建流程中完成第六阶段即驱动安装阶段的部分测试环境重建操作。第七芯片测试配置字段可以用于在测试环境重建流程中完成第七阶段即容器启动阶段的部分测试环境重建操作。第八芯片测试配置字段可以用于在测试环境重建流程中完成第八阶段即容器内环境设置阶段的部分测试环境重建操作。目标关键信息可以是待检测芯片的相关硬件设备信息,例如可以包括但不限于芯片设备ID和数量等信息。

[0127] 图6是本发明实施例四提供的一种八阶段测试环境重建流程的示意图。在一个具体的例子中,如图6所示,八阶段测试环境重建流程可以主要包括下载核心转储文件、下载软件包、系统环境检查、芯片硬件检测、宿主机环境设置、驱动安装、容器启动以及容器内环境设置等各个阶段。芯片测试装置对于芯片复现的每个测试场景均可以采用上述八阶段测试环境重建流程来实现。

[0128] 相应的,如图6所示,在第一阶段的下载核心转储文件的流程中,所需的第一芯片测试配置字段可以为execution.corefile字段,该字段为一个查询字段,可以用于查询核心转储文件。在第一阶段中,如果分阶段环境构建模块确定corefile(核心转储文件)字段非空,表示芯片测试通用流水线中发生了崩溃,此时,分阶段环境构建模块可以解析corefile字段的URL,使用命令行工具或wget下载核心转储文件到对应测试工作空间的工作目录。下载完成后,分阶段环境构建模块还可以验证下载的核心转储文件的完整性,例如

可以检查文件大小和MD5(Message-Digest Algorithm 5 ,消息摘要算法 5)校验码等。第一阶段的返回值为true(成功)或false(下载失败),该阶段可以为后续测试过程中在使用GDB(GNU Debugger,GNU调试器)出现崩溃分析时提供核心转储文件。

[0129] 相应的,如图6所示,在第二阶段的下载软件包的流程中,所需的第二芯片测试配置字段可以为software.download_cmds,该字段可以为表示用于下载软件命令的字段。在第二阶段中,分阶段环境构建模块可以读取软件包列表即download_cmds(下载命令序列)列表,按顺序执行每个下载命令,从而按顺序依次下载各软件包。在下载前可验证下载的软件包是否存在。第二阶段的返回值为true(所有命令成功)或false(任一命令失败),该阶段可以下载芯片的SDK、工具链以及测试数据等各种类型的软件包。

[0130] 相应的,如图6所示,在第三阶段的系统环境检查的流程中,所需的第三芯片测试配置字段可以为hardware.system_check_cmd字段,该字段为表示硬件系统检查命令的字段,可以用于检查系统环境。在第三阶段中,分阶段环境构建模块可以读取system_check_cmd(执行系统状态检查命令)列表,以执行每个检查命令,验证系统挂载点以及内核版本等依赖。如果分阶段环境构建模块确定任一检查失败,则中止环境构建过程并报告错误。第三阶段的返回值为true(检查通过)或false(检查失败),该阶段可以确保宿主机系统满足各类芯片测试的基础依赖,如特定内核版本和挂载点等。

[0131] 相应的,如图6所示,在第四阶段的芯片硬件检测的流程中,所需的第四芯片测试配置字段可以为hardware.target_check_cmd字段,该字段为表示检查硬件目标的状态或性能的字段,可以用于检查芯片硬件信息。在第四阶段中,分阶段环境构建模块可以读取target_check_cmd(芯片检测命令)列表,执行各项芯片检测命令,验证目标芯片是否存在,并解析芯片检测命令的输出值,以提取设备ID和数量等目标关键信息。如果检测失败(芯片设备不存在),分阶段环境构建模块可以中止环境构建过程并提示用户。第四阶段的返回值为true(检测到芯片)或false(未检测到芯片),该阶段是芯片测试场景的专用步骤,通用测试框架不包含此类硬件检测逻辑。例如,该阶段可以实现通过lspci命令检测特定厂商ID的设备。

[0132] 相应的,如图6所示,在第五阶段的宿主机环境设置的流程中,所需的第五芯片测试配置字段可以为environment.host_env_setup字段,该字段为表示在主机环境中设置环境变量和配置的字段,可以用于设置宿主机环境。在第五阶段中,分阶段环境构建模块可以读取host_env_setup(环境设置命令)列表,以按顺序执行每个环境设置命令,更新宿主机环境变量以及创建目录等。第五阶段的返回值为true(所有命令成功)或false(任一命令失败),该阶段可以为芯片驱动安装准备宿主机环境,如设置库路径以及创建必要目录等。

[0133] 相应的,如图6所示,在第六阶段的驱动安装的流程中,所需的第六芯片测试配置字段可以为software.kmd_install_cmd字段,该字段为表示内核模式驱动(KMD)安装命令的字段,可以用于设置宿主机环境。在第六阶段中,分阶段环境构建模块可以读取kmd_install_cmd(内核模式驱动安装命令)字符串,以执行驱动安装命令,安装芯片的内核驱动模块(KMD),验证驱动是否加载成功,例如可以通过lsmod(用于列出当前已加载到内核的所有模块的命令)命令检查驱动模块。如果驱动安装失败,分阶段环境构建模块可以中止环境构建过程并报告错误。第六阶段的返回值为true(驱动安装成功)或false(安装失败),该阶段是芯片测试场景的核心步骤,kmd_install_cmd字段可以包含驱动安装的完整参数,如

“--driver”表示只安装驱动,“-y”表示自动确认,“--no-auto-load”表示不自动加载驱动。

[0134] 相应的,如图6所示,在第七阶段的容器启动的流程中,所需的第七芯片测试配置字段可以为software.docker_launch_cmd字段,该字段为表示启动一个Docker容器的命令的字段,可以用于设置启动容器。在第七阶段中,分阶段环境构建模块可以读取docker_launch_cmd(容器启动命令)列表,按顺序执行每个容器启动命令,等待容器启动完成,如可以通过docker ps(Docker命令行中用于列出当前正在运行的容器的命令)命令检查容器状态确定容器是否启动完成。同时,分阶段环境构建模块可以记录容器ID,供后续命令使用。第七阶段的返回值为ContainerID(容器实例ID),该阶段可以创建隔离的测试环境,挂载芯片设备如AI加速卡设备、设置网络模式以及挂载SDK目录等。

[0135] 相应的,如图6所示,在第八阶段的容器内环境设置的流程中,所需的第八芯片测试配置字段可以为environment.container_env_setup字段,该字段为表示设置容器启动时的环境变量的字段,可以用于设置容器启动时的环境变量。在第八阶段中,分阶段环境构建模块可以读取container_env_setup(容器内环境设置命令)列表,为每个命令添加容器执行前缀,容器执行前缀可以从其他容器相关的字段获取,容器执行前缀例如可以为“docker exec -it {container}”等。分阶段环境构建模块可以按顺序执行每个容器内环境设置命令,以在容器内安装SDK、配置库路径以及安装Python依赖等,并验证环境设置是否成功,如检查关键文件是否存在。第八阶段的返回值为true(所有命令成功)或false(任一命令失败),该阶段可以在容器内完成芯片SDK安装以及环境变量设置等最后的准备工作。

[0136] 可选的,分阶段环境构建模块可以采用状态检查点(Checkpoint)机制,该机制可以记录各阶段完成状态,并基于记录的完成状态支持多种跳过功能。示例性的,状态检查点机制可支持跳过驱动安装、跳过系统检查、跳过宿主机设置、跳过调试符号以及跳过核心转储等跳过功能,从而在重建测试环境的过程中有效跳过已完成步骤和操作,有效提高芯片测试效率。

[0137] 综上,分阶段环境构建模块自下载核心转储与下载软件包起,经系统检查、硬件检测、宿主机环境、驱动安装、容器启动与容器内环境等各个阶段构建对应的测试环境,最终进入测试执行。分阶段环境构建模块构建的各阶段与芯片测试配置文件的相关配置字段对应,支持检查点机制,每个阶段完成后写入job_status.json文件,已完成的阶段在再次执行时自动跳过,支持断点续传。同时支持如跳过驱动安装、跳过系统检查以及跳过宿主机设置等多跳过项选项。

[0138] S550、在所述重构测试环境中根据目标测试命令和目标测试用例自动化执行对应的流水线测试任务。

[0139] 其中,所述目标测试命令和所述目标测试用例基于所述芯片测试配置字段获取。目标测试命令可以是适用于流水线测试任务进行测试的命令。目标测试用例可以是能够对流水线测试任务进行测试的测试用例。

[0140] 示例性的,如图2所示,芯片测试装置的核心处理层还可以配置有测试执行模块(也可以称为用例管理模块),用于管理测试用例的执行,支持正则匹配、环境变量覆盖和交互模式,负责执行芯片测试配置文件中的tests列表、进入容器交互模式(enter)以及与测试执行相关的功能。

[0141] 在本发明的一个可选实施例中,所述在所述重构测试环境中根据目标测试命令和目标测试用例自动化执行对应的流水线测试任务,可以包括:从所述芯片测试配置字段中筛选芯片测试关联字段,并确定用例操作方法;根据所述流水线测试任务的任务类型从所述芯片测试关联字段中筛选所述目标测试命令,并从所述用例操作方法中筛选所述目标测试用例;采用所述目标测试命令根据所述目标测试用例自动化执行对应的流水线测试任务;在自动化执行对应的流水线测试任务的过程中,存储管理测试执行日志;其中,所述用例操作方法包括运行测试用例方法、进入容器交互模式方法、显示命令序列方法以及显示崩溃堆栈方法。

[0142] 其中,芯片测试关联字段可以为与芯片测试相关的字段。

[0143] 在本发明实施例中,可选的,测试执行模块可以用于执行测试执行步骤,管理用例操作方法和维护日志记录器。具体的,在执行测试执行步骤的过程中,测试执行模块可以针对标准化的芯片测试配置文件解析得到的芯片测试配置字段中涉及测试执行的参数进行筛选,确定芯片测试关联字段。示例性的,芯片测试关联字段可以包括receipt_keyword字段、test_prefix字段、replace_test_regex字段以及overrides字段。其中,receipt_keyword字段对应芯片测试配置文件中的字段名,固定值“tests”,用于指示从芯片测试配置文件的tests字段提取测试命令列表。test_prefix字段表示测试命令的公共前缀,用于在每个测试命令前自动添加前缀。例如,如果test_prefix="cd /workspace",tests=["pytest test_a.py"],则实际执行的测试命令为"cd /workspace && pytest test_a.py"。replace_test_regex字段表示测试用例过滤正则表达式,可通过efdbg run命令的“-R”或“--tests-regex”(用于指定仅执行测试名称匹配给定正则表达式的测试用例的参数)参数指定,用于只执行匹配该正则表达式的测试用例。示例性的,如果replace_test_regex="test_conv",tests=["pytest test_conv.py", "pytest test_pool.py"],则只执行"pytest test_conv.py"。overrides字段表示环境变量覆盖列表,可通过efdbg run命令的“-O”或“--override”(用于强制覆盖默认配置、测试参数或行为的参数)参数指定,用于覆盖容器内的环境变量,其执行方式为在测试命令前添加export(导出)语句。

[0144] 测试执行模块可以管理多种类型的用例操作方法,且可以解析流水线测试任务的任务类型,进而根据具体的任务类型从芯片测试关联字段中筛选所需的目标测试命令,并从用例操作方法中筛选所需的目标测试用例,以采用目标测试命令根据目标测试用例自动化执行对应的流水线测试任务。

[0145] 在一个具体的例子中,测试执行模块可以提供运行测试用例的方法cmd_run,该测试执行方法对应efdbg run <config_file> [options]命令,cmd_run方法的执行流程如下:

步骤1:调用场景管理模块的设置命令cmd_setup()设置环境。如果要设置的测试场景已存在且环境已构建,则可以跳过环境构建步骤;否则,执行八阶段测试环境重建流程。

[0146] 步骤2:提取芯片测试配置文件中的tests列表。具体的,测试执行模块可以从芯片测试配置文件的tests字段读取测试命令列表。

[0147] 步骤3:如果用户指定了“--tests-regex”参数,则过滤匹配的测试。具体的,测试执行模块可以使用正则表达式匹配测试命令,只保留匹配的测试。例如,如果正则表达式为

“conv”,则只保留“pytest test_conv.py”的测试。

[0148] 步骤4:在容器内依次执行测试命令。具体的,测试执行模块可以为每个测试命令添加test_prefix前缀,使用容器执行前缀按顺序执行每个测试命令,实时输出测试日志到控制台和日志文件。

[0149] 步骤5:记录执行日志。具体的,可以将所有测试命令和输出记录到运行日志run_log文件,记录每个测试的退出码,如成功或失败。

[0150] cmd_run方法的返回值可以为测试结果对象,可以包括但不限于测试总数、成功数、失败数以及日志路径等内容。

[0151] 在一个具体的例子中,测试执行模块可以提供进入容器交互模式的方法cmd_enter,该方法对应efdbg enter [<config_file>]命令,cmd_enter方法的执行流程如下:

步骤1:检查容器是否存在。具体的,测试执行模块可以从job_status.json读取container字段,执行“docker ps -a”命令检查容器状态。如果容器不存在或已停止,测试执行模块可以报告错误。

[0152] 步骤2:获取工作目录。测试执行模块可以从job_status.json读取发布目录的release_dir字段,切换到工作目录,便于用户访问配置文件和日志等。

[0153] 步骤3:执行docker exec(用于在已运行的容器内部执行新命令的指令)进入容器。测试执行模块可以执行“docker exec -it {container} bash命令”进入容器的交互式shell(shell指的是命令行解释器及其脚本语言,用于自动化执行测试流程中的系统级操作)。用户可以手动执行调试命令、查看文件以及运行GDB等。用户通过退出命令退出shell后,容器继续运行。

[0154] cmd_enter方法的返回值为交互式会话ShellSession。

[0155] 在一个具体的例子中,测试执行模块可以提供显示命令序列的方法show_commands,该方法对应efdbg show -c命令,show_commands方法的执行流程如下:

步骤1:遍历所有构建步骤。测试执行模块可以读取八阶段测试环境重建流程中执行过的所有命令,如可以从日志文件中提取命令记录。

[0156] 步骤2:生成可执行的shell脚本。测试执行模块可以将所有命令按执行顺序组织成一个可执行的bash脚本。同时添加注释,标注每个阶段的名称,如“#阶段1:下载核心转储”等,输出到控制台或保存为脚本文件。

[0157] show_commands方法的返回值为string,内容为命令序列文本,可以帮助用户理解环境构建的完整流程,或手动执行某些步骤进行调试。

[0158] 在一个具体的例子中,测试执行模块可以提供显示崩溃堆栈的方法show_backtrace,该方法对应efdbg show -b命令,show_backtrace方法的执行流程如下:

步骤1:检查corefile目录。具体的,测试执行模块可以检查工作目录下是否存在核心转储文件。如果不存在,提示用户“未发现核心转储文件”。

[0159] 步骤2:对每个核心转储文件执行gdb bt -(backtrace 命令的简写,用于显示当前线程的函数调用栈)命令,执行gdb <program> <corefile> -batch -ex “bt”命令。gdb <program> <corefile> -batch -ex “bt”命令为在自动化测试中用GDB非交互式加载可执行文件 <program> 和核心转储文件 <corefile>,并自动打印调用栈后立即退出的命令。进一步的,测试执行模块可以提取崩溃堆栈,解析GDB输出,提取崩溃位置和函数调用栈等

关键信息,格式化输出到控制台。

[0160] `show_backtrace`方法没有返回值,直接输出到控制台,可以快速查看崩溃堆栈,无需手动启动GDB。

[0161] 在本发明实施例中,测试执行模块还可以通过日志记录器管理测试执行日志。其中,测试执行日志可以是流水线测试任务在测试过程中所生成的日志。示例性的,日志记录器可以通过`log_dir`字段管理日志目录路径,所有日志文件都可以存储在该目录下;通过`setup_log`字段管理环境构建的日志文件路径,用于记录八阶段环境构建的所有命令和输出;通过`run_log`字段管理测试执行日志文件路径,用于记录测试用例的执行命令、输出和退出码。示例性的,日志记录器还可以通过写入日志行`write(line)`方法将日志内容追加到日志文件,同时可以输出到控制台;通过根据类型返回对应日志文件的路径`get_logger_path(type)`方法生成带时间戳的日志文件路径,为每次流水线测试任务的测试执行生成独立的日志文件,便于追溯。

[0162] 综上,测试执行模块可以提供测试过滤机制,具体通过正则表达式匹配测试用例,实现灵活的测试子集执行;支持在不修改配置文件的情况下,临时调整测试环境,实现环境变量的覆盖;通过相关方法进入容器,支持手动调试和问题分析,完成交互式调试;通过相关方法生成可重复执行的脚本,导出命令序列,便于手动复现。

[0163] 在一个具体的例子中,以下是测试执行模块的形式化定义:

```
TestCaseManagerModule {
  // 测试执行步骤
  StepTest {
    receipt_keyword: "tests"
    test_prefix: string           // 测试命令前缀
    replace_test_regex: string   // 测试用例匹配正则表达式
    overrides: list              // 环境变量覆盖列表
  }
  // 用例操作
  operations {
    // 运行测试用例(对应run命令)
    cmd_run(receipt: string) → TResult {
      1. 调用cmd_setup()设置环境
      2. 提取配置中的tests列表
      3. 如有test_regex,过滤匹配的测试
      4. 在容器内依次执行测试命令
      5. 记录执行日志
    }
    // 进入容器交互模式(对应enter命令)
    cmd_enter() → ShellSession {
      1. 检查容器是否存在
      2. 获取工作目录
```

```

        3. 执行docker exec进入容器
    }
    // 显示命令序列(对应show -c命令)
    show_commands() → string {
        1. 遍历所有构建步骤
        2. 生成可执行的shell脚本
    }
    // 显示崩溃堆栈(对应show -b命令)
    show_backtrace() → void {
        1. 检查corefile目录
        2. 对每个core文件执行gdb bt
    }
}
// 执行日志
Logger {
    log_dir: string           // 日志目录(workspace/logs/)
    setup_log: string         // 设置日志
    run_log: string           // 运行日志
    write(line) → void
    get_logger_path(type) → string // 生成带时间戳的日志路径
}
}

```

在本发明的一个可选实施例中,上述芯片测试方法还可以包括:获取各所述测试场景的流水线测试任务的任务实时运行状态信息;其中,所述任务实时运行状态信息包括配置文件路径、容器名称以及环境构建各阶段的完成信息;所述实时运行状态信息用于在自动化执行各所述测试场景的流水线测试任务的过程中对已完成阶段进行跳过处理。

[0164] 示例性的,如图2所示,芯片测试装置的核心处理层还可以配置有状态管理模块,维护各工作场景的构建与运行状态,为场景管理模块和环境构建模块提供状态查询与更新接口。可选的,各测试场景的流水线测试任务的任务实时运行状态信息以job_status.json文件形式存储于各场景工作目录下,记录当前场景的配置文件路径、容器名称以及环境构建各阶段的完成情况等信息。分阶段环境构建模块在执行各阶段前通过状态管理模块查询is_done(已完成),已完成阶段可直接跳过;完成后调用更新方法写入状态。场景管理模块在列表和展示时也可以从状态管理模块处读取状态用于展示;配置解析模块不直接读写状态,仅提供解析结果供上述模块使用。如图2所示,状态管理模块与场景管理模块、分阶段环境构建模块以及测试执行模块并列,通过共享工作目录下的job_status.json与上述模块协同,实现断点续传与状态展示。

[0165] 可选的,场景管理模块与状态管理模块可以共同实现测试工作场景的隔离与生命周期的管理。图7是本发明实施例四提供的一种测试工作场景生命周期状态机的结构示意图。在一个具体的例子中,如图7所示,以jobid标识的测试工作场景经创建后进入已创建

(CREATED) 状态,通过run命令执行测试时进入运行中 (RUNNING) 状态;运行过程中可通过暂停pause()命令挂起进入已暂停 (PAUSED) 状态,并可通过继续执行命令resume()恢复;通过进入enter命令可进入容器内部的交互模式 (INTERACTIVE) 进行手动调试,退出(exit)后恢复;最终可通过删除clean命令主动删除,或由自动清理策略(auto_clean_up)按过期天数回收,进入已销毁 (DESTROYED) 状态。场景管理模块通过各工作空间独立的job_status.json完整记录并控制上述状态机流转。

[0166] 如图7所示,测试工程场景经创建后进入可运行状态,用户可通过list里边命令查看、select选择命令切换当前场景,通过clean命令删除。同时,可以在后台按过期天数自动清理过期场景。每个场景对应独立工作目录,与图6中八阶段流水线的执行目录一致。job_status.json文件可以存于该目录下,由状态管理模块维护,记录各构建阶段完成情况,实现与图6中各阶段检查点的对应。上述list、select、show以及clean等CLI命令为场景管理模块的调用入口。

[0167] 在本发明的一个可选实施例中,芯片测试装置可以提供以下标准化命令行接口:

```
# ===== 环境设置命令 =====
efdbg setup <receipt.json> [options]
# 功能:根据配置文件设置调试环境
# 参数:receipt.json - 芯片测试配置文件路径或URL
# 选项:
#   -n, --dry-run      不实际执行,仅打印命令
#   -v, --verbose      打印详细信息
#   --skip-kmd          跳过驱动安装
#   --skip-check        跳过系统检查
#   --skip-host         跳过宿主机环境设置
#   --skip-debug        跳过调试符号加载
#   --skip-corefile     跳过核心转储处理
# ===== 测试执行命令 =====
efdbg run [<receipt.json>] [options]
# 功能:设置环境并运行测试用例
# 参数:receipt.json-芯片测试配置文件路径(可选,默认使用最近的配置)
# 选项:
#   -R, --tests-regex   按正则表达式匹配运行测试
#   -O, --override      覆盖测试环境变量
# ===== 容器交互命令 =====
efdbg enter [<receipt.json>]
# 功能:进入当前场景的容器交互模式
# 参数:receipt.json -芯片测试配置文件路径(可选)
# ===== 场景管理命令 =====
efdbg list [-t|--tags] [-v|--verbose]
# 功能:列出所有工作场景
```

```

# 选项:
#   -t, --tags          列出可用的预设标签
efdbg select <index>
# 功能:按索引选择工作场景
efdbg show [<receipt.json>] [options]
# 功能:显示工作场景信息
# 选项:
#   -s, --status        显示任务状态
#   -r, --root          显示根配置
#   --log               显示日志文件列表
#   -b, --backtrace     显示崩溃堆栈(如有)
#   -c, --commands     显示命令序列
# ===== 清理命令 =====
efdbg clean [<receipt.json>] [options]
# 功能:清理工作场景
# 选项:
#   -a, --all           清理所有工作场景
#   -y, --confirm       确认直接删除
#   -o, --outdated     清理过期的工作场景
# ===== 设备管理命令 =====
efdbg reset [devices...]
# 功能:重置AI加速卡设备
# 参数:devices - 设备ID列表,如 0 1 2

```

芯片测试装置的核心功能是基于标准化的芯片测试配置文件的快速复现,即读取标准化的芯片测试配置文件,自动重建芯片测试通用流水线中的测试环境。在一个具体的例子中,下面通过三个典型场景展示芯片测试装置快速复现功能的使用流程:

场景1:使用标准化的芯片测试配置文件一键复现环境。

[0168] 当测试工程师需要在本地复现CI/CD流水线中的测试场景时,只需执行两步:

步骤1:执行读取配置的efdbg setup receipt.json命令,芯片测试装置读取json格式的芯片测试配置文件,自动解析芯片测试配置文件的元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构等五区结构字段,并自动执行八阶段测试环境重建流程:如果corefile字段非空,则下载核心转储文件-下载软件包-系统环境检查-芯片硬件检测-宿主机环境设置-驱动安装-容器启动-容器内环境设置。环境构建完成后,创建一个测试工作空间,其jobid为芯片测试配置文件中的meta.jobid字段值。

[0169] 步骤2(可选):执行其他命令进行调试。例如,执行efdbg enter命令进入容器内进行交互式调试;执行efdbg run receipt.json命令一键执行芯片测试配置文件中的测试用例;执行efdbg show -c命令查看场景执行的完整命令历史等。

[0170] 场景2:对比分析不同版本的测试场景。

[0171] 当测试工程师需要对比两个版本的测试场景的测试环境差异时,可以执行下述流程:

步骤1:设置第一个场景。如通过efdbg setup v1_receipt.json命令创建jobid为v1_test的测试工作场景。

[0172] 步骤2:设置第二个场景。如通过efdbg setup v2_receipt.json命令创建jobid为v2_test的测试工作场景。

[0173] 步骤3:在两个测试工作场景间快速切换。例如,可以通过efdbg select v1_test命令秒级切换到v1场景;通过efdbg select v2_test命令秒级切换到v2场景。

[0174] 步骤4:对比测试工作场景信息。例如,可以通过efdbg show -c命令显示当前测试工作场景的命令序列,并通过对比两个测试工作场景的命令历史,快速定位环境差异。

[0175] 场景3:崩溃问题分析。

[0176] 当CI/CD流水线中出现程序崩溃时,可以执行下述流程:

步骤1:使用标准化的芯片测试配置文件复现崩溃环境。如通过efdbg setup crash_receipt.json命令芯片测试配置文件的corefile字段包含核心转储文件路径,芯片测试装置自动下载核心转储文件到工作目录。

[0177] 步骤2:进入容器分析崩溃堆栈。例如通过efdbg enter命令进入容器,并通过gdb program corefile命令使用gdb分析核心转储文件。

[0178] 步骤3(可选):使用芯片测试装置的快捷命令,如通过efdbg show -b命令自动解析核心转储文件,显示崩溃堆栈摘要。

[0179] 上述快速复现芯片测试场景功能的核心在于对芯片测试配置文件的配置规范的深度理解和解析,特别是对芯片测试配置文件中专属于芯片领域专用字段的自动化执行能力。下面是典型的命令使用示例:

典型命令使用示例

基本复现操作

efdbg setup receipt.json # 读取配置,自动重建完整环境

efdbg run receipt.json # 一键执行测试用例

efdbg enter # 进入容器交互调试

场景管理操作

efdbg list # 列出所有工作场景

efdbg select 1 # 切换到指定场景

efdbg clean --all # 清理所有场景

信息查看操作

efdbg show -c # 显示命令序列

efdbg show -b # 显示崩溃堆栈

efdbg show -s # 显示场景状态

预设标签使用(可选)

efdbg run topsinference:3.6.105 # 使用预设标签快速复现

如图2所示,芯片测试装置的核心处理层还可以按需扩展其他功能模块,如日志管理模块等。核心处理层的配置解析模块、场景管理模块、分阶段环境构建模块、测试执行模

块、日志管理模块与状态管理模块等各功能模块通过工作目录与芯片测试配置文件协同,从而完成配置加载、变量替换、场景隔离、八阶段环境构建以及测试执行与状态持久化。

[0180] 表1列举出了本发明实施例提供的芯片测试装置与现有通用流水线测试状态的技术效果对比结果,示例性的,如表1所示,芯片测试装置实现了输入标准化,接收标准化的芯片测试配置文件作为唯一输入,并可自动完成八阶段环境构建,实现一键重建的功能。同时,芯片测试装置基于状态管理,可以自动跳过已完成的阶段以及实现断点续传功能;基于jobid的工作目录隔离,可以支持多测试场景并行处理,同时可以实现秒级场景切换,无需重新构建测试场景。

[0181] 表1 芯片测试装置技术效果对比列表

效果指标	传统方式	使用本芯片测试装置	说明
基于描述文件复现	无标准方式	efdbg setup 一键复现	直接读取标准化的芯片测试配置文件
环境构建时间	手动操作 30-60 分钟	efdbg setup 自动化 5-10 分钟	八阶段流水线自动执行
学习成本	需要了解完整流水线	只需掌握 setup/run/enter 等命令	屏蔽底层复杂依赖
场景切换	重新配置环境	efdbg select 一键切换(秒级)	多场景并行管理,秒级切换
复现断点续传	每次从头开始	已完成阶段自动跳过	检查点机制支持中断恢复
配置复用	每次手动编写	支持变量替换+本装置预设标签定位	芯片测试配置文件以及测试装置复现形成闭环
崩溃分析	手动收集 corefile	efdbg show -b 自动显示堆栈	复现后的问题诊断支持

[0182] 上述技术方案提供了一种基于标准化的芯片测试领域特定配置规范的配置解析与执行装置,能够理解芯片测试场景的专用字段含义,支持变量替换,提供预设标签快速定位机制,自动执行分阶段环境构建,实现测试场景的精确复现。

[0183] 需要说明的是,本公开所涉及的相关信息(包括但不限于用户设备信息、用户个人信息等)和数据(包括但不限于用于展示的数据、分析的数据等),均为经用户授权或者经过各方充分授权的信息和数据,且相关数据的收集、使用和处理遵守相关地区的相关法律法规和标准。

[0184] 需要说明的是,以上各实施例中各技术特征之间的任意排列组合也属于本发明的保护范围。

[0185] 实施例五

图8是本发明实施例五提供的一种芯片测试装置的示意图,如图8所示,所述装置包括:芯片测试配置文件获取模块810、芯片测试配置字段解析模块820以及流水线测试任务执行模块830,其中:

芯片测试配置文件获取模块810,用于获取标准化的芯片测试配置文件;其中,所述标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区

结构、环境配置区结构以及测试执行描述区结构;

芯片测试配置字段解析模块820,用于对所述标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段;

流水线测试任务执行模块830,用于根据所述芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流水线测试任务。

[0186] 本发明实施例通过获取标准化的芯片测试配置文件,该标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构等多种数据结构,从而可以对标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段,进而根据芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流水线测试任务。上述芯片测试方法可以有效解决现有通用流水线测试工具难以有效理解芯片领域配置信息,以致无法复现芯片测试场景的问题,能够快速且精确地复现芯片的专用测试场景。

[0187] 可选的,芯片测试配置文件获取模块810还用于:采集芯片测试通用流水线在芯片测试过程中各测试阶段的实时状态关联信息;根据所述实时状态关联信息生成标准化的芯片测试配置文件。

[0188] 可选的,芯片测试配置文件获取模块810还用于:根据所述实时状态关联信息填充所述芯片测试配置文件的预设数据结构,得到初始芯片测试配置文件;对所述初始芯片测试配置文件中的配置数据进行规范化验证;在确定所述初始芯片测试配置文件通过验证的情况下,将所述初始芯片测试配置文件进行序列化处理,得到所述标准化的芯片测试配置文件。

[0189] 可选的,芯片测试配置文件获取模块810还用于:从所述实时状态关联信息中提取元数据字段,并根据所述元数据字段填充所述芯片测试配置文件的元数据区结构;从所述实时状态关联信息中提取芯片硬件环境检测字段,并根据所述芯片硬件环境检测字段填充所述芯片测试配置文件的硬件环境描述区结构;从所述实时状态关联信息中提取芯片软件环境字段,并根据所述芯片软件环境字段填充所述芯片测试配置文件的软件环境描述区结构;从所述实时状态关联信息中提取环境配置字段,并根据所述环境配置字段填充所述芯片测试配置文件的环境配置区结构;从所述实时状态关联信息中提取测试执行字段,并根据所述测试执行字段填充所述芯片测试配置文件的测试执行描述区结构。

[0190] 可选的,芯片测试配置字段解析模块820还用于:对所述标准化的芯片测试配置文件的各数据结构进行解析和验证,得到初始化芯片测试配置字段;从所述初始化芯片测试配置字段提取芯片领域专用字段;对所述芯片领域专用字段进行结构化配置,得到所述芯片测试配置字段。

[0191] 可选的,芯片测试配置字段解析模块820还用于:根据所述标准化的芯片测试配置文件的目标数据结构解析得到芯片检测命令字段以及驱动安装命令字段;根据当前测试场景的流水线测试任务自动化检测目标替换变量,并根据所述目标替换变量自动化替换解析得到的字段中的目标占位符;验证必填字段、字段类型、芯片专用字段以及目标依赖关系的规范性。

[0192] 可选的,流水线测试任务执行模块830还用于:对各所述测试场景的流水线测试任务创建并维护独立测试工作空间;在各所述测试工作空间内根据所述芯片测试配置字段执

行八阶段测试环境重建流程,得到各所述流水线测试任务对应的重构测试环境;在所述重构测试环境中根据目标测试命令和目标测试用例自动化执行对应的流水线测试任务;其中,所述目标测试命令和所述目标测试用例基于所述芯片测试配置字段获取。

[0193] 可选的,流水线测试任务执行模块830还用于:根据第一芯片测试配置字段下载核心转储文件并保存至对应测试工作空间的工作目录;根据第二芯片测试配置字段读取软件包列表,并按顺序依次下载各软件包;根据第三芯片测试配置字段读取系统环境检查命令列表,并按顺序依次执行每个系统环境检查命令;根据第四芯片测试配置字段读取并执行芯片检测命令列表,解析芯片检测命令输出值,提取目标关键信息;根据第五芯片测试配置字段读取环境设置命令列表,并按顺序依次执行每个环境设置命令,以更新宿主机环境变量并创建目录;根据第六芯片测试配置字段读取并执行驱动安装命令,以安装芯片的内核驱动模块;根据第七芯片测试配置字段读取并执行容器启动命令列表中的各个容器启动命令,并存储记录容器ID信息;根据第八芯片测试配置字段读取容器内环境设置命令,对各所述容器内环境设置命令添加容器执行前缀后,按顺序执行各所述容器内环境设置命令。

[0194] 可选的,流水线测试任务执行模块830还用于:从所述芯片测试配置字段中筛选芯片测试关联字段,并确定用例操作方法;根据所述流水线测试任务的任务类型从所述芯片测试关联字段中筛选所述目标测试命令,并从所述用例操作方法中筛选所述目标测试用例;采用所述目标测试命令根据所述目标测试用例自动化执行对应的流水线测试任务;在自动化执行对应的流水线测试任务的过程中,存储管理测试执行日志;其中,所述用例操作方法包括运行测试用例方法、进入容器交互模式方法、显示命令序列方法以及显示崩溃堆栈方法。

[0195] 可选的,上述装置还包括实时运行状态管理模块,用于:获取各所述测试场景的流水线测试任务的任务实时运行状态信息;其中,所述任务实时运行状态信息包括配置文件路径、容器名称以及环境构建各阶段的完成信息;所述实时运行状态信息用于在自动化执行各所述测试场景的流水线测试任务的过程中对已完成阶段进行跳过处理。

[0196] 上述芯片测试装置可执行本发明任意实施例所提供的芯片测试方法,具备执行方法相应的功能模块和有益效果。未在本实施例中详尽描述的技术细节,可参见本发明任意实施例提供的芯片测试方法。

[0197] 由于上述所介绍的芯片测试装置为可以执行本发明实施例中的芯片测试方法的装置,故而基于本发明实施例中所介绍的芯片测试方法,本领域所属技术人员能够了解本实施例的芯片测试装置的具体实施方式以及其各种变化形式,所以在此对于该芯片测试装置如何实现本发明实施例中的芯片测试方法不再详细介绍。只要本领域所属技术人员实施本发明实施例中芯片测试方法所采用的装置,都属于本申请所欲保护的范围。

[0198] 实施例六

图9示出了可以用来实施本发明的实施例的电子设备10的结构示意图。电子设备旨在表示各种形式的数字计算机,诸如,膝上型计算机、台式计算机、工作台、个人数字助理、服务器、刀片式服务器、大型计算机、和其它适合的计算机。电子设备还可以表示各种形式的移动装置,诸如,个人数字处理、蜂窝电话、智能电话、可穿戴设备(如头盔、眼镜、手表等)和其它类似的计算装置。本文所示的部件、它们的连接和关系、以及它们的功能仅仅作作为示例,并且不意在限制本文中描述的和/或者要求的本发明的实现。

[0199] 如图9所示,电子设备10包括至少一个处理器11,以及与至少一个处理器11通信连接的存储器,如只读存储器ROM 12、随机访问存储器RAM 13等,其中,存储器存储有可被至少一个处理器执行的计算机程序,处理器11可以根据存储在只读存储器ROM 12中的计算机程序或者从存储单元18加载到随机访问存储器RAM 13中的计算机程序,来执行各种适当的动作和处理。在RAM 13中,还可存储电子设备10操作所需的各种程序和数据。处理器11、ROM 12以及RAM 13通过总线14彼此相连。输入/输出(I/O)接口15也连接至总线14。

[0200] 电子设备10中的多个部件连接至I/O接口15,包括:输入单元16,例如键盘、鼠标等;输出单元17,例如各种类型的显示器、扬声器等;存储单元18,例如磁盘、光盘等;以及通信单元19,例如网卡、调制解调器、无线通信收发机等。通信单元19允许电子设备10通过诸如因特网的计算机网络和/或各种电信网络与其他设备交换信息/数据。

[0201] 处理器11可以是各种具有处理和计算能力的通用和/或专用处理组件。处理器11的一些示例包括但不限于中央处理单元(CPU)、图形处理单元(GPU)、各种专用的人工智能(AI)计算芯片、各种运行机器学习模型算法的处理器、数字信号处理器(DSP)、以及任何适当的处理器、控制器、微控制器等。处理器11执行上文所描述的各个方法和处理,例如芯片测试方法。

[0202] 可选的,芯片测试方法可以包括:获取标准化的芯片测试配置文件;其中,所述标准化的芯片测试配置文件包括元数据区结构、硬件环境描述区结构、软件环境描述区结构、环境配置区结构以及测试执行描述区结构;对所述标准化的芯片测试配置文件的各数据结构进行解析,得到芯片测试配置字段;根据所述芯片测试配置字段采用分阶段的方式自动化执行各测试场景的流水线测试任务。

[0203] 在一些实施例中,芯片测试方法可被实现为计算机程序,其被有形地包含于计算机可读存储介质,例如存储单元18。在一些实施例中,计算机程序的部分或者全部可以经由ROM 12和/或通信单元19而被载入和/或安装到电子设备10上。当计算机程序加载到RAM 13并由处理器11执行时,可以执行上文描述的芯片测试方法的一个或多个步骤。备选地,在其他实施例中,处理器11可以通过其他任何适当的方式(例如,借助于固件)而被配置为执行芯片测试方法。

[0204] 本文中以上描述的系统和技术各种实施方式可以在数字电子电路系统、集成电路系统、场可编程门阵列(FPGA)、专用集成电路(ASIC)、专用标准产品(ASSP)、芯片上系统的系统(SOC)、负载可编程逻辑设备(CPLD)、计算机硬件、固件、软件、和/或它们的组合中实现。这些各种实施方式可以包括:实施在一个或者多个计算机程序中,该一个或者多个计算机程序可在包括至少一个可编程处理器的可编程系统上执行和/或解释,该可编程处理器可以是专用或者通用可编程处理器,可以从存储系统、至少一个输入装置、和至少一个输出装置接收数据和指令,并且将数据和指令传输至该存储系统、该至少一个输入装置、和该至少一个输出装置。

[0205] 用于实施本发明的方法的计算机程序可以采用一个或多个编程语言的任何组合来编写。这些计算机程序可以提供给通用计算机、专用计算机或其他可编程数据处理装置的处理器,使得计算机程序当由处理器执行时使流程图和/或框图中所规定的功能/操作被实施。计算机程序可以完全在机器上执行、部分地在机器上执行,作为独立软件包部分地在机器上执行且部分地在远程机器上执行或完全在远程机器或服务器上执行。

[0206] 在本发明的上下文中,计算机可读存储介质可以是有形的介质,其可以包含或存储以供指令执行系统、装置或设备使用或与指令执行系统、装置或设备结合地使用的计算机程序。计算机可读存储介质可以包括但不限于电子的、磁性的、光学的、电磁的、红外的、或半导体系统、装置或设备,或者上述内容的任何合适组合。备选地,计算机可读存储介质可以是机器可读信号介质。机器可读存储介质的更具体示例会包括基于一个或多个线的电气连接、便携式计算机盘、硬盘、随机存取存储器(RAM)、只读存储器(ROM)、可擦除可编程只读存储器(EPROM或快闪存储器)、光纤、便捷式紧凑盘只读存储器(CD-ROM)、光学储存设备、磁储存设备、或上述内容的任何合适组合。

[0207] 为了提供与用户的交互,可以在电子设备上实施此处描述的系统和技术,该电子设备具有:用于向用户显示信息的显示装置(例如,CRT(阴极射线管)或者LCD(液晶显示器)监视器);以及键盘和指向装置(例如,鼠标或者轨迹球),用户可以通过该键盘和该指向装置来将输入提供给电子设备。其它种类的装置还可以用于提供与用户的交互;例如,提供给用户的反馈可以是任何形式的传感反馈(例如,视觉反馈、听觉反馈、或者触觉反馈);并且可以用任何形式(包括声输入、语音输入或者触觉输入)来接收来自用户的输入。

[0208] 可以将此处描述的系统和技术实施在包括后台部件的计算系统(例如,作为数据服务器)、或者包括中间件部件的计算系统(例如,应用服务器)、或者包括前端部件的计算系统(例如,具有图形用户界面或者网络浏览器的用户计算机,用户可以通过该图形用户界面或者该网络浏览器来与此处描述的系统和技术实施方式交互)、或者包括这种后台部件、中间件部件、或者前端部件的任何组合的计算系统中。可以通过任何形式或者介质的数字数据通信(例如,通信网络)来将系统的部件相互连接。通信网络的示例包括:局域网(LAN)、广域网(WAN)、区块链网络和互联网。

[0209] 计算系统可以包括客户端和服务器。客户端和服务器一般远离彼此并且通常通过通信网络进行交互。通过在相应的计算机上运行并且彼此具有客户端-服务器关系的计算机程序来产生客户端和服务器的关系。服务器可以是云服务器,又称为云计算服务器或云主机,是云计算服务体系中的一项主机产品,以解决了传统物理主机与VPS服务中,存在的管理难度大,业务扩展性弱的缺陷。

[0210] 本发明实施例还公开了一种计算机程序产品,所述计算机程序产品包括计算机程序,所述计算机程序在被处理器执行时实现本发明任一实施例所提供的芯片测试方法。该程序产品与本发明各实施例所公开的芯片测试方法属于相同发明构思,因此不在此赘述。

[0211] 应该理解,可以使用上面所示的各种形式的流程,重新排序、增加或删除步骤。例如,本发明实施例中记载的各步骤可以并行地执行也可以顺序地执行也可以不同的次序执行,只要能够实现本发明实施例公开的技术方案所期望的结果,本文在此不进行限制。

[0212] 上述具体实施方式,并不构成对本公开保护范围的限制。本领域技术人员应该明白的是,根据设计要求和因素,可以进行各种修改、组合、子组合和替代。任何在本公开的精神和原则之内所作的修改、等同替换和改进等,均应包含在本公开保护范围之内。

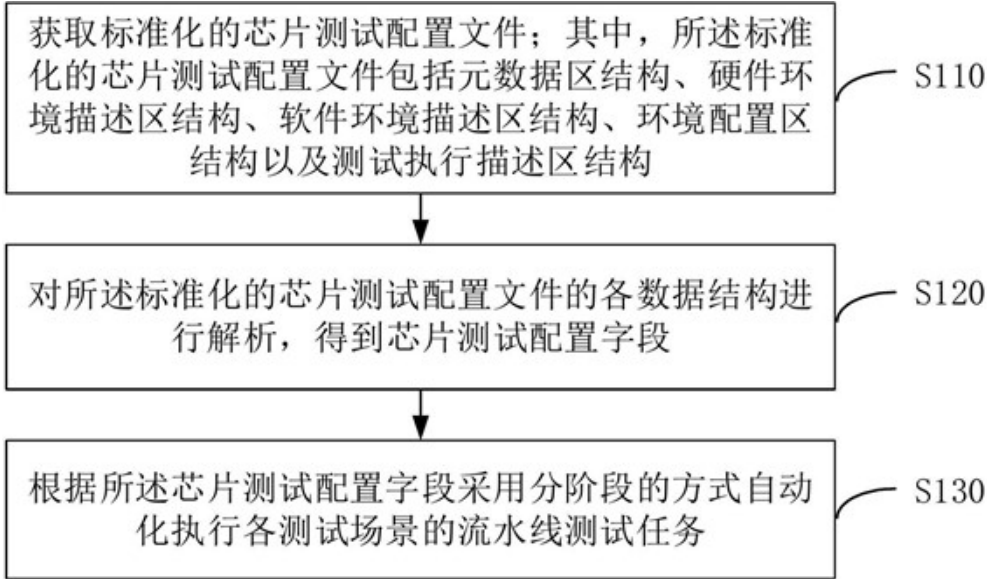


图1

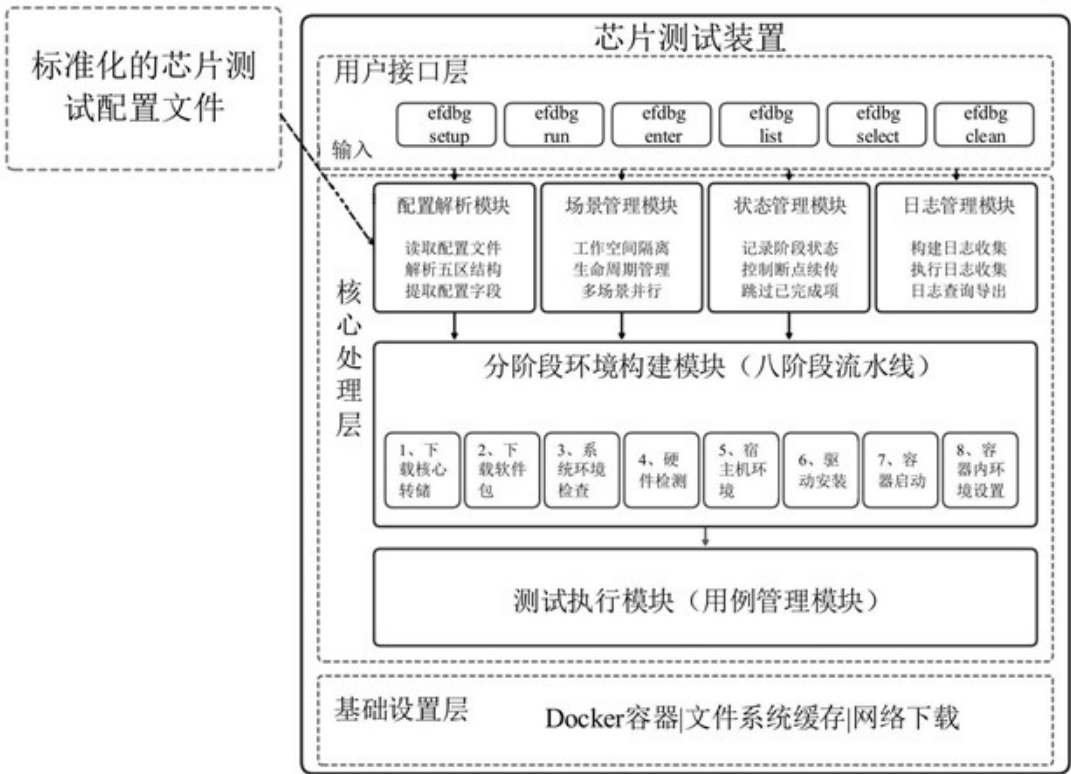


图2

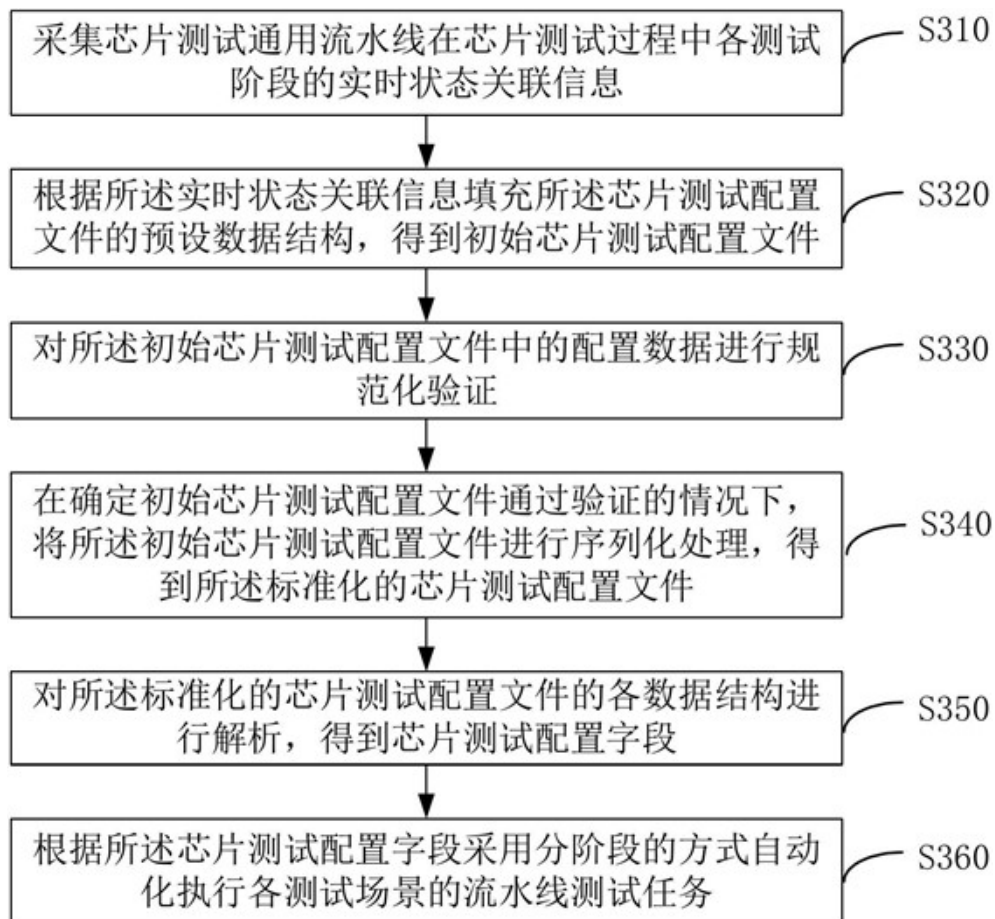


图3

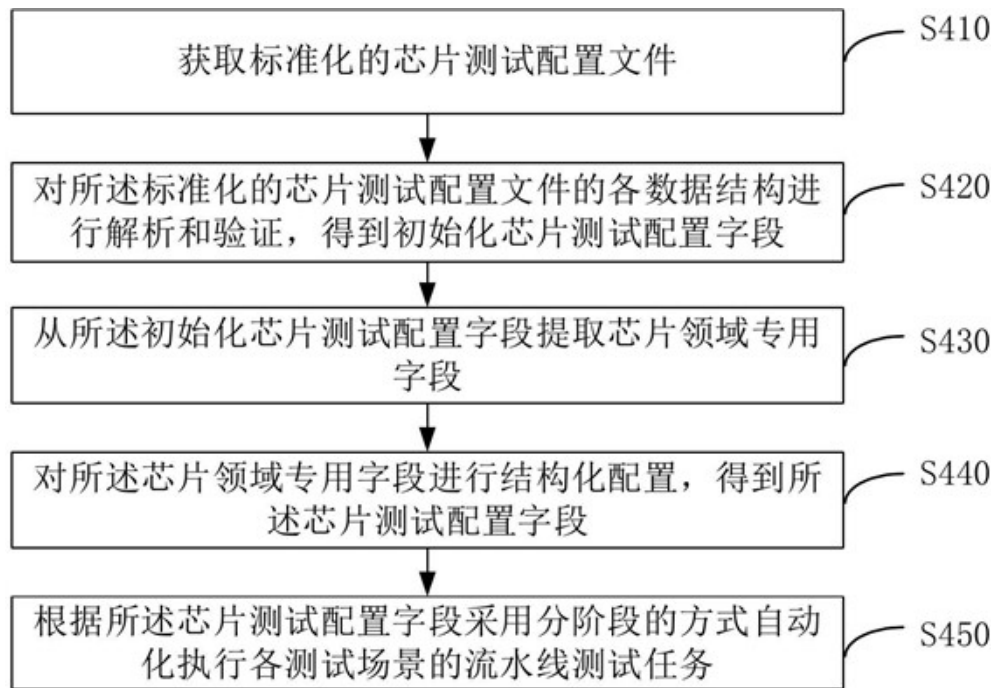


图4

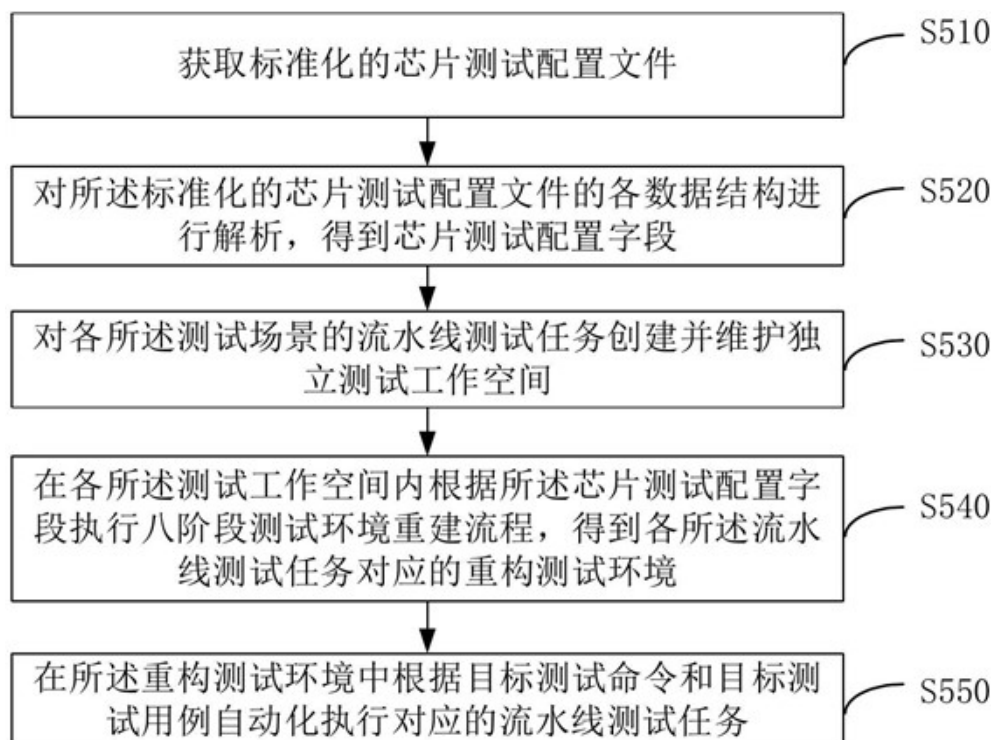


图5

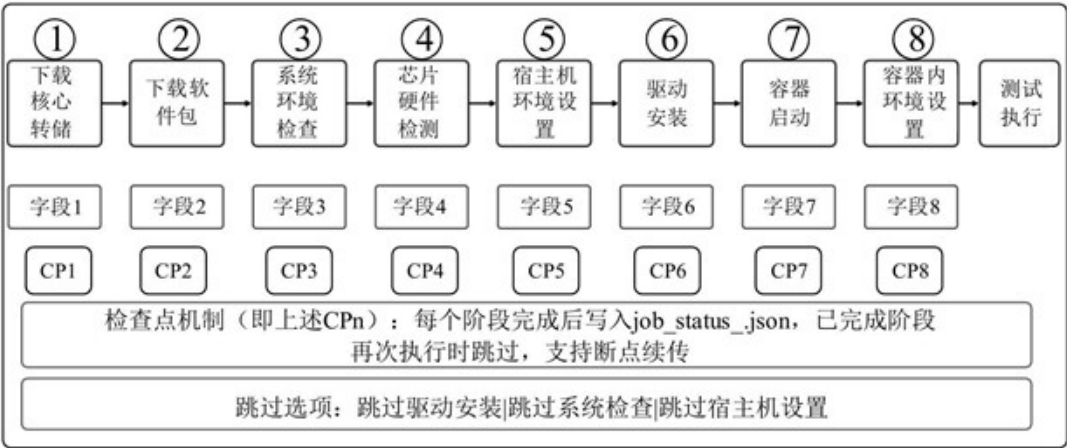


图6

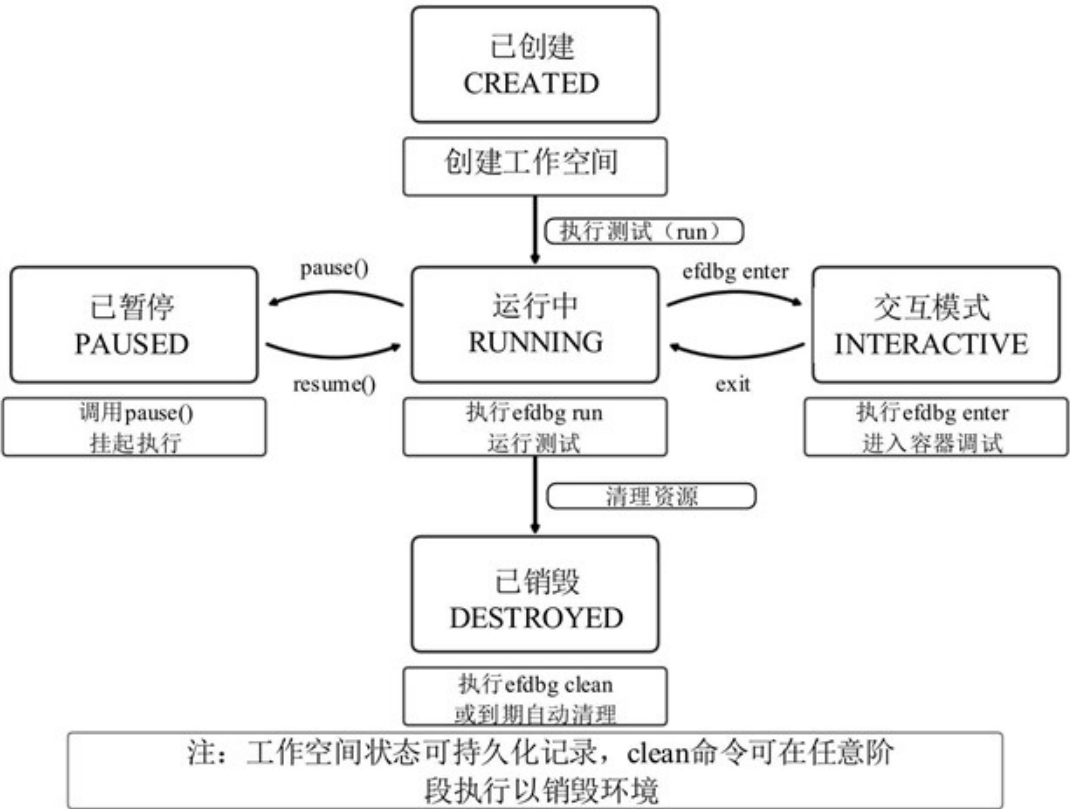


图7

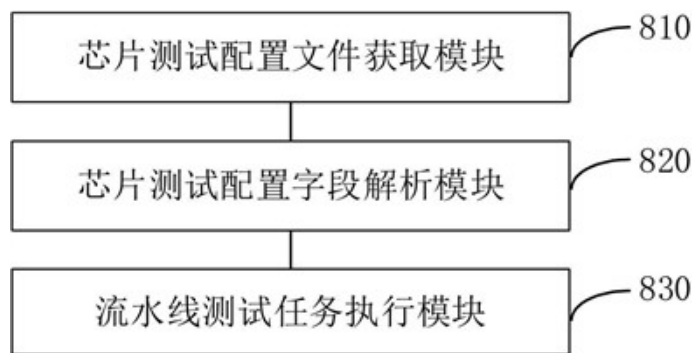


图8

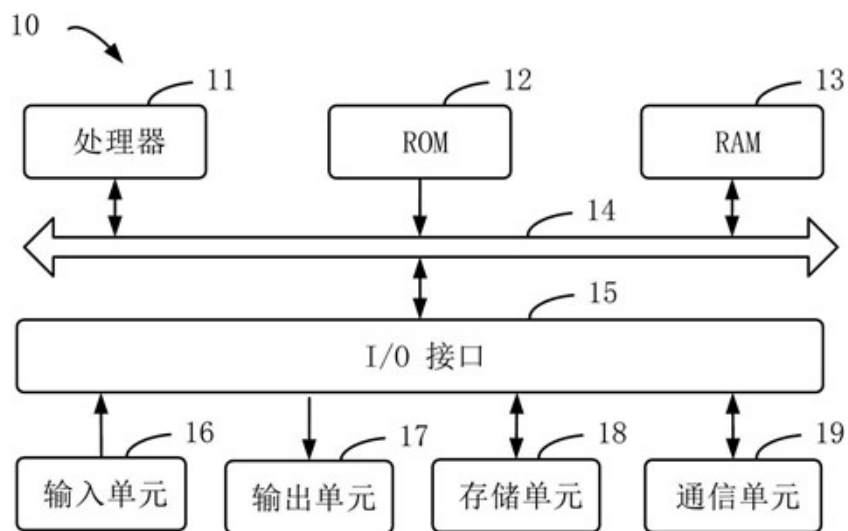


图9